

Chapter 8. Data Storage Layer

In the previous chapter, you learned how the data ingestion layer works, including the mechanisms, technologies, and formats used to ingest data into a financial data infrastructure. Once ingested, data must be stored and persisted in a storage location for further processing and querying. This is where the data storage layer comes into the picture.

To help you understand how to build a robust storage layer, this chapter will provide you with the necessary fundamentals and concepts, along with illustrations of technologies and their applications in finance. First, you'll learn how to approach the design of a data storage system (DSS) using appropriate criteria. Next, the concept of a data storage model (DSM) and its categorization criteria will be introduced. Then, I will present a comprehensive list of DSSs relevant to the financial industry, highlighting each DSS's key features, data modeling concepts, technical implementations, and financial applications.

Principles of Data Storage System Design

Throughout this book, I use the term *data storage system* (DSS) to denote a software implementation that enables the storage and retrieval of data. In many cases, people use the term “database” to refer to a storage solution. However, databases are only one type of DSS, albeit a popular one.

As a financial data engineer, knowing how to assess, choose, design, and implement a DSS should be one of your primary skills and areas of knowledge. The DSS is a core component in several financial applications, such as trading, payment, and messaging platforms. Making the wrong DSS choice can be quite costly and lead to a notable impact on the perfor-

mance and reliability of your infrastructure. If a change is required later, you might find yourself dealing with a complex and expensive data migration project that wastes resources and time. This is particularly true when other applications and layers have already been built on top of the existing DSS.

Designing the appropriate DSS may look overwhelming at first glance. This is because of the vast number of technologies, patterns, constraints, business requirements, and marketing materials that affect the decisions around a DSS. To address this challenge, this section will provide a set of universally applicable principles that can guide your DSS design and implementation strategy.

Principle 1: Business Requirements

Although data engineering appears to be a strictly technological discipline, the truth is that it is highly driven by business requirements and customer needs. I highly recommend that you do not proceed with the design of a DSS without first including the business team's feedback into your decision-making process. As a financial data engineer, you don't necessarily need to be a business expert, but by knowing just enough to understand business expectations, you are likely to make the best design choices.

Business requirements can vary in terms of complexity (e.g., what and how many features are required?), flexibility (are they strict requirements?), technical feasibility (are there technical limitations?), predictability (can you anticipate the requirements in advance?), timing (have the requirements already been formulated?), and stability (do requirements change with time?).

Examples of data-related business requirements include the following:

- Ease of data access (e.g., a user interface)
- Schema flexibility (e.g., a new feature requires a new field in the data schema)
- User scalability (e.g., a high number of concurrent users)
- Speed of data access (e.g., for high-frequency trading)

- Querying capabilities (e.g., complex analytical queries and filters)
- Storage needs (e.g., storing a massive dataset such as market transactions)
- Data sharing (e.g., to collaborate with external companies)
- Data aggregation (e.g., for regulatory reporting)

DSS design and business requirements are matched through an iterative and collaborative approach. To succeed in this process, financial data engineers need to communicate in business terms and translate the requirements into technical implementation. Along the way, they need to spot all possible bottlenecks or technical limitations and reach an agreement with the business team to achieve feasibility.

Principle 2: Data Modeling

Data modeling is a crucial, yet often underestimated, practice in DSS design. In general, data modeling focuses on how data is organized, connected, and stored to meet both technical and business requirements. Importantly, the data engineering community still lacks a widely recognized data modeling framework. Nevertheless, market practitioners have typically relied on a popular approach that breaks down data modeling into three sequential phases: *conceptual*, *logical*, and *physical*. These phases are developed as one progresses from business needs to a comprehensive data storage specification, which is the desired end state.

The conceptual phase is technology independent and involves a communicative process where data engineers and stakeholders together discuss all their data needs. Think of the conceptual phase as a whiteboard session where various members of your team meet to discuss the initial data requirements. The main outcome of this phase is identifying a model for data, relationships, constraints, and querying needs that are essential to the business application. At this stage, no decisions are made about how the data will be stored or which DSS will be used for this purpose.

Subsequently, during the logical phase, the conceptual model is mapped to a blueprint of structured and technical constructs such as rows, columns, tables, and documents that a DSS can implement. At the end of the

logical phase, you still haven't picked a specific DSS, but you should clearly know how the data will appear in the DSS and what DSM to use (for example, relational or document models). Later in this chapter, we will discuss DMSs in detail.

Finally, the physical phase translates the logical model into the DSS language, which is often known as *Data Definition Language* (DDL). This stage's main outcome involves choosing a specific DSS (e.g., a relational database management system) and developing a clear understanding of how data is stored (e.g., on disk, in memory, or hybrid), and how data is replicated, sharded, partitioned, or distributed.

DATA MODELING USE CASE IN FINANCE: REFERENCE DATA AND FINANCIAL STANDARDS

In [Chapter 2](#), we discussed the well-known problem of financial reference data, i.e., metadata used to describe financial instruments. The main challenge with reference data is the lack of a universally accepted framework for its representation and formatting, especially for complex financial instruments like derivatives. This issue is an excellent example of a financial domain problem that can be addressed through data modeling. Data modeling offers a robust solution by providing structured and standardized conceptual models for representing, collecting, and storing reference data. For an excellent treatment of this topic, I recommend Robert Mamayev's book *Data Modeling of Financial Derivatives: A Conceptual Approach* (Apress, 2013), in which he illustrates how to structure and describe derivatives such as futures, forwards, options, swaps, and forward rate agreements using advanced data modeling techniques.

Similarly, the development and formulation of financial standards like ISO 20022 can be viewed through the lens of data modeling. As discussed in [Chapter 7](#), creating an ISO 20022 message model involves conceptual, logical, and physical stages, mirroring the methodology outlined in this section.

An important thing to keep in mind is that data modeling is an iterative process. Data engineers and business teams may continually reorganize,

restructure, and optimize data models to fit new or revised business needs.¹

Principle 3: Transactional Guarantee

One of the most important features of a DSS is its ability to ensure data consistency and reliability. In other words, the data must always accurately reflect its true state. For example, if you have \$10,000 in your bank account and purchase a book for \$50, your new balance should be \$9,950, not \$9,000. Similarly, if you purchase a book for \$50 and at the same time you also purchase a car for \$10,000, then you either end up with \$9,950 or \$0 but not -\$50.

This feature is known as a *transactional guarantee*. Here, “transaction” describes an internal DSS mechanism that allows the bundling together of multiple instructions into a single, all-or-nothing operation.² In the data technology landscape, transactional guarantees are commonly implemented using two main models: *ACID* and *BASE*.

ACID is the most popular model,³ and it stands for:

Atomicity

A DSS transaction either succeeds completely (gets committed) or, in case of a fault, gets entirely aborted, reverting the DSS to the state before the transaction started. In our previous book and car example, atomicity would mean that either both items are bought (a book and a car) or neither is bought and the account balance remains intact. A violation of atomicity would occur if we attempted to purchase both the book and the car in a single transaction, where the purchase of the book succeeds but the car purchase fails.

Consistency

DSS transactions need to preserve structural integrity and enforce defined constraints. Consider again our previous example of a \$10,050 transaction with an available balance of \$10,000. Assume that the account balance has a non-negative constraint. In this scenario, the transaction should fail to prevent the balance from be-

coming negative, ensuring consistency. Maintaining this consistency is typically the responsibility of DSS engineers, who establish and implement data consistency checks, constraints, and validations based on business requirements, both at the DSS level as well as within the applications that interact with it.

Isolation

Concurrent DSS transactions get executed in isolation from one another while ensuring integrity and resolving conflicts. For instance, if you attempt two separate transactions—one to buy a book and another to buy a car—and only one can succeed, the DSS identifies this conflict and ensures proper resolution. Maintaining isolation is primarily the responsibility of the DSS. A notable strategy is *snapshot isolation*, where each transaction within the DSS perceives a consistent snapshot of data that includes all committed changes up to the transaction's initiation.⁴

Durability

The DSS preserves and persists the committed transactions even in case of a system failure (e.g., a power outage or crash). In our example, suppose you just completed the book purchase for \$50. If the transaction is completed and you receive a notification of success, then the DSS must ensure that your purchase has been recorded in nonvolatile storage such as a hard drive or SSD. One of the most reliable techniques for ensuring durability is *Write-Ahead-Logging* (WAL), whose main idea is to first “log” all changes to be applied to the data before persisting them to disk.

ACID properties are of primary importance for the design of financial DSSs. Examples include financial transaction processing, where money or ownership of financial instruments is transferred between customer accounts, and order matching systems that execute buy and sell orders. In all of these applications, it's critical to maintain a consistent state of the data by ensuring that transactions either complete entirely or not at all. Throughout this chapter, I will provide detailed technical illustrations of how various DSSs implement and ensure ACID properties.

WARNING

Many data storage solution providers make the ACID compliance promise. However, they don't all implement it in the same manner. Some may use a portion of the ACID properties (e.g., atomicity) or develop their own version of ACID. When designing a financial application with strict requirements in terms of data consistency and a transactional guarantee, ensure you thoroughly comprehend the ACID features of the chosen solution. If not, this might influence your product's market reputation.

In some cases, enforcing ACID compliance may impact performance or may not even be that necessary. In other cases, ACID might even be impractical to achieve. This is where a lighter version of ACID, called BASE, has been introduced.

BASE stands for:

Basically available

Instead of enforcing immediate consistency, BASE-modeled DSSs focus on ensuring availability by distributing and replicating data across a set of nodes in a data storage cluster. In the event of a failure in one node, data would still be available through another node that holds a replica of the data.

Soft state

As the requirement of immediate consistency is relaxed, the BASE model delegates the responsibility of achieving data consistency to the engineer instead of the DSS itself. In other words, it is the developer's problem to ensure data consistency, and it is no longer a feature of the DSS itself.

Eventually consistent

Even though BASE does not aim for immediate consistency, this does not mean that it never achieves it: it is eventually achieved, meaning that data will converge to a consistent state at some point in the future. No guarantee is made, however, about when consistency will be achieved.

The BASE model is quite useful for designing distributed and big data storage solutions, especially for analytical purposes where speed and scalability are more important than strict consistency. For instance, a trading platform might analyze vast amounts of data to provide traders with insights and market trends. In this scenario, the BASE model could be suitable, as it prioritizes processing large datasets quickly, even if some data might be slightly outdated.

Principle 4: Consistency Tradeoffs

A distributed DSS consists of several machines, or nodes, working together to store and manage data collectively. Designing and building such systems can be highly complex. Several consistency tradeoff theorems have been formalized to assist engineers with this challenge. The most prominent is the CAP theorem, which stands for consistency, availability, and partition tolerance. According to the CAP theorem, if a *network partition* occurs, where some nodes in a distributed DSS are unable to communicate due to a network failure, the system can guarantee at most two out of the following features:

Consistency

All users always see the same latest version of the data, regardless of which node they interact with.

Availability

The DSS responds to all user requests at all times, though it may not always provide the most recent data.

Partition tolerance

The data storage system continues to operate even when there is a network partition.

NOTE

Keep in mind that consistency in ACID is different from consistency in the CAP theorem. In the CAP theorem, consistency means that all nodes maintain a consistent view of the data. Conversely, in the context of ACID, consistency ensures that the database remains in a valid and correct state.

To illustrate the CAP theorem with an example, suppose we have a bank with two ATMs connected over a network.⁵ Assume the bank requires that customer balances never drop below zero, but there is no central database system to ensure this condition. Instead, a copy of the database is stored on both ATM instances, and as users carry out their operations on their accounts, the two ATMs communicate over the network to ensure consistency.

Now, suppose that a network partition happens, and the two ATMs can no longer communicate. In such a case, you, as the designer of the DSS, need to decide whether to prioritize consistency or availability. If you prioritize consistency, you are likely to refuse all ATM operations (withdrawal, deposit, balance checks) until the partition is resolved. This guarantees balance consistency, but the ATM services will not be available to customers. If, on the other hand, you favor availability, then you may allow each ATM to perform deposit and withdrawal operations but at the risk of ending in an inconsistent state until the partition is resolved (e.g., withdrawing the entire balance from both ATMs, ending up with a negative balance).

Keep in mind that the CAP theorem is quite simplistic and basic. In real applications, tradeoffs might be more complex than just 100% availability or 100% consistency. For example, in our ATM example, rather than completely blocking or allowing all ATM operations, the bank may still allow for balance inquiry or small money withdrawals during a partition and only refuse large withdrawals and deposits.

As an extension of the CAP theorem, the PACELC theorem was introduced. PACELC considers two system scenarios: first, if the system has a network partition, it must decide how to trade off consistency and availability; sec-

ond, if the system is running normally without a network partition, it must still decide how to trade off consistency and latency. The latency/consistency tradeoff exists only if the distributed DSS replicates data.⁶

When working with distributed DSSs, several of them offer the option to tune the consistency level. For example, Azure Cosmos DB offers five levels of consistency guarantees, which, from strongest to weakest, are strong, bounded staleness, session, consistent prefix, and eventual; Amazon DynamoDB offers either strong or eventual consistency levels; and Cassandra offers several consistency levels that also range from weak to strong.

Principle 4: Scalability

Scalability is a highly desired property of DSSs. Generally, a system is considered scalable if it can effectively handle varying or increasing amounts of workload. Importantly, scalability can be achieved differently depending on the component or layer of the DSS. To simplify, we can categorize DSS scalability into two main aspects: storage scalability and compute scalability.

A DSS achieves storage scalability if it can seamlessly store increasing volumes of data without hitting a space limitation. In more practical terms, storage scalability means that the system can store data at any scale: gigabytes, terabytes, or even petabytes.

Conversely, compute scalability in a DSS refers to its ability to efficiently handle varying data read and write requests. A common measure of compute scalability is the maximum number of concurrent read/write requests that the DSS can accommodate.

Scalability can be achieved in multiple ways. One approach is vertical scalability, where a single machine is replaced with a bigger one that has more storage, CPU, and RAM. On the other hand, horizontal scalability requires a distributed system that scales by adding a new node to an existing cluster of connected nodes.

When designing a DSS, it is crucial to evaluate its capability to handle anticipated workloads. This is often done via stress testing and benchmarking. A database stress test works by simulating the generation of a large amount of data, queries, and concurrent requests while observing how the DSS behaves in terms of response time, errors, and reliability. The goal, in lay terms, is to stress the DSS to identify its operational limits and where it may encounter system failures or breakpoints.

A *database benchmark* is a special type of database stress test that relies on well-defined and industry-accepted testing methodology. Examples include [Transaction Processing Performance Council \(TPC\) standards](#) such as TPC-C and TPC-E.⁷ TPC-C simulates an order-entry environment where a population of users concurrently submit a variety of transactions against a database. Transactions may vary in complexity and include placing and fulfilling orders, keeping track of payments, confirming order status, and checking the level of stock in the warehouse. TPC-C evaluates performance based on the number of new-order transactions processed per minute.

TPC-E is a more sophisticated benchmark that simulates a brokerage firm receiving and fulfilling various customer transactions related to trades, account inquiries, and market research. To execute client requests, the brokerage firm interacts with financial markets and updates account information. Performance is measured using the transactions per second (TPS) metric.

A best practice is to conduct both industry benchmark tests and internally defined ad hoc tests to gain insights into specific behaviors of the DSS that may not be fully addressed by standardized benchmarks. Commercial database vendors frequently employ this dual approach to showcase the robustness and reliability of their products.⁸

Principle 5: Security

Ensuring strong database security is crucial in financial applications. When designing for security, a variety of processes, tools, and controls

can be put in place to protect a DSS against malicious or accidental threats. These include, for example, the following:

- Data encryption at rest (e.g., customer account information residing in a database)
- Data encryption in transit (e.g., when transmitting payment information over a network)
- Database access permissions and roles
- Separation of database servers from other application servers
- Backups and data recovery plans
- Real-time security information and event monitoring

Data security features vary across different DSSs. Therefore, it's essential to stay informed about your institution's specific security requirements and ensure that the chosen DSS can effectively meet them.

Data Storage Modeling

To implement a DSS, a large number of technological choices are available. However, comparing such technologies is not trivial. Some technologies share several features, yet others may be based on noncomparable design principles. Additionally, on occasion, even the lines between the different data technologies might become blurry as they introduce more of the same features. To this end, this book will take a different approach by relying on the concept of *data storage models* (DSMs) rather than data storage technologies.

A DSM refers to the logical design and structure of a DSS, which determines the manner in which data can be stored, modeled, optimized, accessed, and manipulated. The concept of a DSM is closely related to the general practice of data modeling discussed earlier. To better contextualize things, consider a DSM as a focused subset within data modeling that specifically addresses the logical data modeling phase.

DSMs are technology agnostic, allowing them to be implemented using a variety of technologies, with the possibility of employing the same technology for implementing different DSMs. As a result, conceptualizing in

terms of DSM is considerably easier and less susceptible to technology-specific limitations.

In today's data engineering landscape, a variety of DSMs are available. In the following sections, I will cover eight DSMs in depth, highlighting their main features and applications in finance. But first, let's illustrate a number of popular criteria that are often used to categorize DSMs.

SQL Versus NoSQL

If you are part of the data engineering community, you often hear discussions comparing NoSQL and SQL. However, many argue that the SQL versus NoSQL comparison lacks a clear technical basis. But to illustrate the main idea, I will briefly explain its origin.

In data engineering, the most trusted type of DSSs have traditionally been relational database management systems, or more simply, SQL databases (illustrated in detail in the section [“The Relational Model”](#)). With the massive increase in data volume, variety, and velocity, traditional SQL databases hit scalability and performance limitations. In a nutshell, SQL databases could only scale vertically, which is a limited scaling strategy since it is constrained by the maximum capacity of a single server.

To overcome this issue, people began exploring more scalable and high-performance alternatives. This led to the development of a new family of database technologies such as document, graph, key-value, and wide-column databases. To distinguish these database technologies from traditional SQL databases, they were grouped under the label NoSQL, which stands for “Not only SQL.” NoSQL databases tackled the issue of scalability by adopting a horizontal scaling approach that relies on distributed systems and partitioning principles.

Importantly, SQL databases adhere to internationally recognized SQL standards, which have contributed to their reliability and trust within the data engineering community. In contrast, NoSQL databases are diverse and lack standardized behavior or common standards. Moreover, to ensure high performance, some NoSQL databases sacrifice several of the core features that gained SQL popularity, such as integrity constraints,

ACID transactions, and advanced querying capabilities. Keep in mind, however, that NoSQL is not always more performant than SQL; it mostly depends on the function for which it is used.⁹ Additionally, it is worth noting that later developments in database technologies made it possible for SQL databases to scale horizontally, thus reducing the gap between SQL and NoSQL.¹⁰

Primary Versus Secondary

A primary DSS is designed to act as the secure and permanent repository for data storage.

Think of the primary DSS as the stronghold of your data. But in many cases, the primary DSS may not have all the features needed to interact with the data. This is where a secondary DSS comes into the scene. A secondary DSS reads and stores a copy of data from a primary database and allows users to perform more advanced data querying and filtering operations.

A secondary DSS may not maintain the same level of data consistency as the primary DSS, but it is generally intended for use cases where this discrepancy does not cause significant issues.

A secondary DSS is typically utilized in situations where the primary DSS stores massive amounts of log or text data, which might be valuable if analyzed using sophisticated search queries. In this scenario, a search engine-like secondary DSS (e.g., Elasticsearch) may be implemented to regularly fetch and index log data from the primary DSS (more on Elasticsearch in [“The Document Model”](#)). Every time you see an additional monitoring dashboard bundled with an application such as an orchestration engine or a managed database, it’s very likely that a secondary database is used for this purpose.¹¹

Operational Versus Analytical

A DSS can be designed to handle different types of business processes. In data engineering, a distinction is often made between operational and an-

alytical processes. Operational processes refer to the day-to-day operations and transactions within a business. Examples may include the following:

- Find customer account details.
- Update the account balance.
- Record and track financial transactions.
- Execute a payment or fund transfer orders.

Analytical processes, on the other hand, are concerned with understanding what is going on within a business; for example:

- Assess the performance of the trading desk.
- Monitor the effect of different investment strategies.
- Understand the main cost and profit drivers.

DSSs intended to handle operational processes are often called *online transaction processing* (OLTP), while those designed for analytical processes are known as *online analytical processing* (OLAP). Generally speaking, OLTPs prioritize reliability and transactional guarantee, while OLAPs favor speed and advanced querying capabilities. A typical scenario involves the use of OLTP as a primary DSS for business transactions and an OLAP as a secondary DSS that stores a copy of OLTP data for analytical purposes.

Native Versus Non-Native

A DSS can serve a single function or a variety of functions. For instance, it might be designed for storing and querying tabular data exclusively, or it could handle document, graph, and key-value data as well. Crucially, various types of design optimizations may be required for supporting each function. This is where a relevant distinction is often drawn between *native* and *non-native* DSSs.

To illustrate the difference, let's say we want a DSS to store and query graph data. A DSS that is specifically designed and optimized for graph data (e.g., Neo4j) is called *native*, whereas a DSS that is capable of han-

dling graph data but was not primarily designed for graphs in mind (e.g., PostgreSQL) is called non-native.¹²

Choose a native DSS when your application’s core functionality relies on an optimized DSS tailored for specific tasks (for example, choosing a native graph DSS for a network analysis application).

Multi-Model Versus Polyglot Persistence

Having in mind the difference between native and non-native DSSs, another important distinction is often made between multi-model and polyglot persistence DSS patterns. Polyglot persistence uses different native DSSs for each function. For instance, you can use PostgreSQL for storing user account information data, Neo4j for managing social network data and graph-based queries, MongoDB for storing and querying user-generated content and posts, and Redis for caching frequently accessed data for improved performance.

Alternatively, a multi-model DSS supports multiple DSMs within a single integrated environment. Such a system may offer support for different DSMs either natively or through extensions. For example, PostgreSQL is a native relational DSS, but it also offers extensions for geospatial data (PostGIS) and image data (PostPic). On the other hand, solutions such as Microsoft Azure Cosmos DB offer native support for document, graph, and key-value data models.

The polyglot persistence approach offers better performance, but it may add significant overhead. In contrast, the multi-model is much simpler but may end up behaving like the proverbial “Jack of all trades, master of none,” which means that while it can accomplish a lot, it won’t provide outstanding performance at any one function.

NOTE

Keep in mind that data storage technologies are continuously evolving and adding new features and functionalities. If a given data storage technology doesn't support a specific feature today, it doesn't mean it won't tomorrow. A missing feature is often an optimization choice that can be changed in a future release.

Data Storage Models

Now that you understand what a DSM is, let's explore the various types of DSMs you might encounter when building a DSS for financial data.

Remember, these models are not mutually exclusive. Financial institutions often use a mix of DSMs to construct DSSs for a range of operational and analytical purposes.

The Data Lake Model

A data lake serves as a centralized repository capable of storing massive amounts of data in raw, unprocessed formats. Data lakes are the most flexible type of DSS since they allow for reliable and cheap storage of any type of data: big or small, structured or unstructured, static or stream. In a typical scenario, data lakes are used to store large amounts of miscellaneous data, including files, news text, documents, logs, data snapshots, archives, and backups.

Why data lakes?

Data lakes provide several features that financial institutions often seek. These include the following:

Data variety and agility

Data lakes offer a flexible storage solution to ingest, store, and analyze any type of data without the need to define a schema or enforce a structure. Data lakes are said to implement *schema on read*, meaning that the data schema is generated on the fly during data query. This is in contrast to *schema on write*, where data is first modeled and structured before being stored.

Simple data ingestion

Data ingestion into a data lake is simple and cost-effective because it does not necessitate transformations or harmonization, thereby lowering data ingestion expenses.

Data integration

Data lakes are useful for organizations that want to consolidate all their data in one central place. This can be practical for purposes such as compliance, data aggregation, risk management, and customer analysis, as well as experimenting with new ideas and datasets. Moreover, it eliminates data silos within the organization.

Data archiving

Data lakes offer a cost-effective solution for data archiving and retention.

Data analysis and insights

Data lakes store data in their original raw format. This allows organizations to perform ad hoc data querying and transformation and access historical snapshots of data.

Data governance

Data lakes can be designed with data governance practices, including access control, cataloging, auditing, reporting, and compliance.

Separation of storage and compute

Data lakes can scale to accommodate large volumes of data and access patterns. If you implement a computation layer on top of a data lake, this means that you have a separation of storage and compute layers. With such separation, you can scale and manage storage and computation independently.

It's important to remember, however, that data lakes are not query-optimized in the same manner as relational databases or data warehouses. Furthermore, performance may be impacted by a data lake's growing size or deeply nested hierarchical structure.

Technological implementations of data lakes

As a DSM, a data lake is an abstract idea that conceptualizes a centralized repository for storing raw data, independent of any specific technology. If a data lake aligns with your use case, the next step would involve selecting a technology to implement it. The classical solution has traditionally been the *[Hadoop Distributed File System \(HDFS\)](#)*. HDFS is a part of the Hadoop ecosystem, which is extensively used to build big data applications. It is an open source, distributed, scalable, and fault-tolerant file storage solution for working with large amounts of data. It runs on commodity servers (mostly on premises), thus making it a cost-effective solution. It is written in Java and has several configuration options that can match various business needs. In addition to HDFS, other prominent open source data lake solutions include MooseFS, Ceph, and GlusterFS.

With the emergence of the cloud, a major preference shift occurred toward building data lakes using managed cloud storage solutions. Examples include AWS's Simple Storage Service (S3), Azure Blob Storage, Google Cloud Storage, and DigitalOcean Spaces. Cloud-based data lakes are considerably easier to build and use; they scale seamlessly and require minimal configuration and maintenance. You can even create a basic cloud data lake in less than a minute if you already have an account!

Nevertheless, don't get overexcited with the simplicity of cloud storage solutions. When designing an enterprise-wide data lake solution, quite a few challenges and factors need to be considered to avoid building a monster. Two things worth discussing in this regard are data modeling and data governance, which I will cover in more detail in the next two sections.

Data modeling with data lakes

When designing data lakes, data modeling is rarely a major discussion point. The reason is that data lakes are not assumed to enforce a specific schema or structure on the ingested data. Even though this might be a valid point, data lakes can still have their own data models and architecture. To illustrate how data modeling works for data lakes, let's take a

cloud-based data lake solution such as AWS S3 as our desired technological implementation.

To start with data lake data modeling, bucket architecture is the first thing to consider. Buckets are the main containers of objects stored in S3. Think of buckets as the logical data model for your data lake, which are distinguished based on your business requirements and can be assigned user permissions independently of the other buckets. For example, one bucket may be dedicated to storing log data, another for financial vendor data, and a third for analytics data. Alternatively, it is common to organize buckets into [zones](#), such as landing zones for raw files, staging zones for enriched files, business zones for analytics and research data, and trusted zones for anonymized and analysis-ready data.¹³

TIP

There is no single recipe for how and how many buckets to create, but I can recommend keeping in mind architecture simplicity: don't create too many buckets such that you won't know which data is where, and don't create one huge bucket to store all kinds of data.

The second element of data lake modeling is a folder structure that organizes data items based on their categories and relationships. The most common folder structure is the tree-like hierarchical structure where parent folders contain subfolders, which in turn may contain other subfolders and so on. A simplistic zone-based folder structure might look like this:

Landing zone

Stores files at the second level, e.g.,
/landing_bucket/year/month/day/hour/minute/second

Staging zone

Cleans and aggregates the data at the daily level, e.g.,
/staging_bucket/year/month/day

Business zone

Splits daily data based on business areas, e.g.,
/business_bucket/business_area/year/month/day

In the landing zone, raw files have schema on read rather than schema on write. This is because they are stored as they arrive without any modification. As data gets transformed in the staging and business zones, it is a good practice to implement schema on write, meaning that you enforce data schema into the files. If you want to apply anonymization to the data, you may want to create a trusted zone where you store data that has been anonymized for safe use and analysis.

A good practice when organizing your data lake folder structure is to avoid having different file formats within the same folder. For example, assume that the ingestion layer loads files in gzip, CSV, and TXT formats. To model this properly, consider a landing zone structure like the following:

- gzip files: */landing_bucket/gzip/year/month/day/hour/minute/second*
- CSV files: */landing_bucket/csv/year/month/day/hour/minute/second*
- TXT files: */landing_bucket/txt/year/month/day/hour/minute/second*

By design, a data lake may accommodate any type of data from any source. All you have to do is specify a bucket name and drop data in it. However, this simplicity can potentially lead to several issues. Think of the *Downloads* folder on your PC; if you are like me, then it is simply a big dump of files of all types and formats; you don't know what's in there, whether it is in the right place, and how to navigate and search for files. Similarly, without the right controls in place, data lakes can easily turn into *data swamps*: an out-of-control place for dumping any data from any source without any checks or validation rules. To avoid this issue, you need to implement a data governance layer on top of a data lake. Let's discuss this topic in more detail.

Data governance

Creating and using a data lake might sound very easy: (1) log in to AWS, (2) select S3, (3) create a bucket, (4) drop files there! This works, and I did it many times. However, architecting a reliable and scalable enterprise-wide data lake is a much more challenging task than this. Large data lakes, in particular, are likely to pose significant issues to users, especially when it comes to understanding things like the following:

- Metadata (information about the data)
- Sources and movements of the data
- Transformations and changes applied to the data
- The architecture of the data (buckets, folders, formats, etc.)
- The level of data quality, consistency, and integrity
- Data access and retrieval methods, which are less intuitive than just using SQL

To address these issues, it is recommended to implement a data governance layer on top of the data lake. Common considerations to keep in mind include the following:

Privacy

Data coming from various sources may include sensitive user information, such as personally identifiable information (PII).

Therefore, the data lake should be designed to anonymize and store this data in compliance with predefined privacy rules. For instance, you can separate buckets that store sensitive data from those that store anonymized data.

Security

Data lakes must be protected against malicious attacks, data loss, and unauthorized access. This is especially important given the extensive use of cloud-based data lakes, which are vulnerable to the risks associated with cloud misconfiguration, i.e., the improper or incorrect setup of cloud resources, services, or settings. Policies to achieve this include encrypting data both at rest and in transit, implementing access controls to specify who can read, upload, and

delete files, and establishing data retention rules, such as security locks, to ensure that archived data is never deleted.

Quality controls

Data must be checked for quality issues as it gets ingested into the data lake. Note, however, that quality controls may not be needed for all types of data. For instance, while log data generally does not require quality checks, business and financial data do benefit from them.

Data cataloging

A data catalog is a valuable tool that provides users of a data lake with functionalities to search for data, metadata, versions, ingestion history, and other attributes associated with the data stored within the data lake.

Importantly, there is no one-size-fits-all approach to implementing a data governance layer on top of a data lake. I suggest beginning with the four elements previously mentioned (privacy, security, quality control, and data cataloging). Keep in mind that for complex data lakes, data governance might be a substantial investment. Some cloud services, such as AWS Lake Formation, provide data lake solutions that include built-in data governance capabilities.

Continuous technological advancements are consistently enhancing the reliability of data lakes. For example, the *data lakehouse* model was created to introduce more structure into data lakes. The main idea of a data lakehouse is to create an integrated data architecture that combines a data lake, data warehouse (covered later in this chapter), and related analytics and processing services.

For example, using AWS, you can build an architecture where you upload and store raw CSV files in an S3 bucket (data lake) and query them with SQL queries using services such as Amazon Athena or Amazon Redshift Spectrum. Another example is Snowflake, which allows users to interact with data stored in a data lake of choice using the Snowflake compute layer. A recommended approach to achieve this is by storing the data in Apache [Iceberg tables](#) that use the Apache Parquet format. Apache

Iceberg is an open source table format for massive analytic datasets, supporting features such as ACID transactions, schema evolution, partitioning, and table snapshots. Similarly, technologies such as [Apache Hudi](#) were introduced to bring database and data warehouse capabilities such as ACID transactions and record-level data updates/deletes to data lakes.

Financial use cases of data lakes

Data lakes have [attracted significant attention within the financial industry](#). Many financial institutions are increasingly investing in building their own data lakes, often leveraging managed cloud storage solutions.

Several reasons can explain the financial sector's excitement for data lakes. An important factor is cloud migration. As many financial institutions are moving to the cloud, data lake solutions such as AWS S3 appear to be particularly appealing because they are simple to set up and can store any type or size of data.

Second, financial institutions process a large amount of unstructured data, such as reports, prospectuses, client files, news data, logs, and more. Data lakes offer an efficient and reliable solution for storing, sharing, and archiving such data.

The third factor is data aggregation, which is both a regulatory requirement and an enabler of innovation. By consolidating data in one location, financial institutions can quickly respond to regulatory inquiries that demand aggregated information, such as total risk exposure. Moreover, emerging trends in data-driven product development and analytics require data from diverse sources, often scattered across decentralized silos within financial institutions. Data lakes enable these institutions to seamlessly consolidate and aggregate data from various sources and silos into a centralized, accessible, and scalable repository.

Fourth is compliance. Financial institutions have a lot of compliance requirements that relate to data privacy, security, retention, and protection. Data lakes may offer a number of desired compliance-related features. For example, AWS S3 [offers Write-Once-Read-Many \(WORM\) storage fea-](#)

tures such as S3 Glacier Vaults and S3 Object Lock to ensure the integrity of archived data in accordance with [US SEC and FINRA rules](#). A number of features are also available to ensure security. For example, S3 provides the *Block Public Access* feature, which simply blocks all public access to S3 buckets. In this case, financial institutions deploy their S3 data lake within a VPC and enable access through [VPC endpoints for Amazon](#).

NASDAQ is a multinational financial services firm operating several financial market platforms worldwide, particularly the NASDAQ Stock Exchange. NASDAQ manages the matching of buy and sell operations on a massive scale and at a rapid speed while providing data-feeding services for its prices and quotes. As a result, a large number of daily records are generated, including orders, quotes, trades, and cancellations. By 2020, NASDAQ was processing around 70,000 records on a daily basis. These records need to be loaded for reporting and billing purposes before the market opens the following day.

To support its large-scale data needs, NASDAQ [invested in a data lake solution leveraging AWS S3](#). This strategic decision enabled NASDAQ to decouple the compute layer from the storage layer, allowing independent scaling of each component. Using S3, NASDAQ gained the capability to manage numerous concurrent read and write operations on large datasets seamlessly, without encountering contention issues.

Using AWS Identity and Access Management (AWS IAM), NASDAQ established a comprehensive access control policy for data stored on S3. Additionally, NASDAQ took advantage of the lower costs of archival storage offered through Amazon S3 Glacier. Moreover, NASDAQ leveraged the [Amazon S3 Object Lock feature](#) to protect objects from deletion or modification, ensuring compliance.

For the compute layer, NASDAQ chose [Amazon Redshift Spectrum](#), a service that allows users to perform SQL queries on data stored in Amazon S3 buckets. The resulting architecture is a data lakehouse that combines a data lake (S3) with a data warehouse (Redshift Spectrum), each scalable independently to support various levels of storage and computing.

The Relational Model

The popular and highly trusted DSSs are built on the relational DSM, which organizes data in rows and columns, forming a table or relation.

These tables can be related to each other through a common attribute (column), hence the term “relational.” [Figure 8-1](#) illustrates the concept.

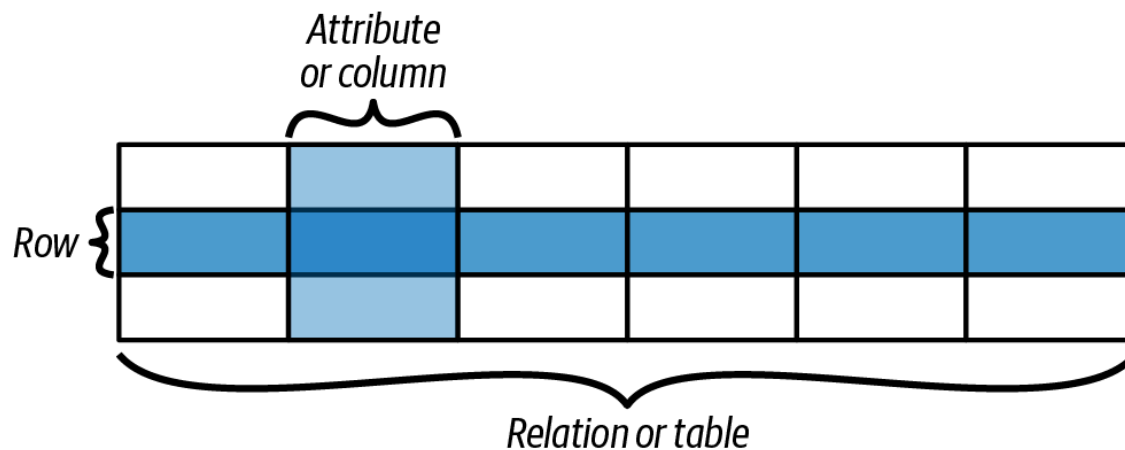


Figure 8-1. Components of the relational DSM

A collection of tables, along with other objects such as views, indexes, stored procedures, and functions, within a relational model is often referred to as a *schema*. Multiple schemas can coexist within the same *database*. For this reason, it is common to refer to a relational DSS as a *relational database management system* (RDMS) or relational database.

Interaction with a relational database happens through a *declarative* language known as *Structured Query Language*, or more commonly SQL. For this reason, relational databases are also commonly called SQL databases.

The relational DSM foundations were laid out by IBM’s expert Edgar Codd in the 1970s. After extensive research on the relational model, Codd [proposed a list of 12 rules](#) that must be observed to develop a true relational database.

Why relational databases?

Relational databases are widely trusted and heavily relied upon by financial market participants, often serving as the default choice for data storage. Let’s explore the reasons behind this preference.

SQL standards

The origins of the SQL language go back to 1974 when Donald Chamberlin and Raymond Boyce from IBM released the essay “SEQUEL: A

Structured English Query Language.” Chamberlin and Boyce relied on Codd’s framework to develop an intuitive and user-friendly language to interface with relational databases.¹⁴

As the language gained wide popularity and a strong reputation among market participants, the *American National Standards Institute* (ANSI) intervened in 1986 to develop and promote a standard for SQL. The ANSI Database Technical Committee (ANSI X3H2) within the Accredited Standards Committee (ASC) X3 developed the first SQL standard, known as ANSI X3.135-1986. In 1987, the ISO started developing an international version of SQL standards, further solidifying SQL’s presence on the global stage. Since then, the standard has been defined and revised under [ISO/IEC 9075](#). What ISO/IEC 9075 offers is a set of features and requirements that need to be implemented in order for a database to become SQL compliant. [Figure 8-2](#) illustrates the historical evolution of SQL standards.

Thanks to ISO/IEC 9075, the SQL language has evolved along a path of continuous improvement, resulting in a rich array of features that continue to make SQL a favored choice among engineers and financial markets to this day. One thing to keep in mind is that SQL standards are advisory, not mandatory technical specifications. It is quite possible to create an SQL database system that implements only a subset of SQL standards.

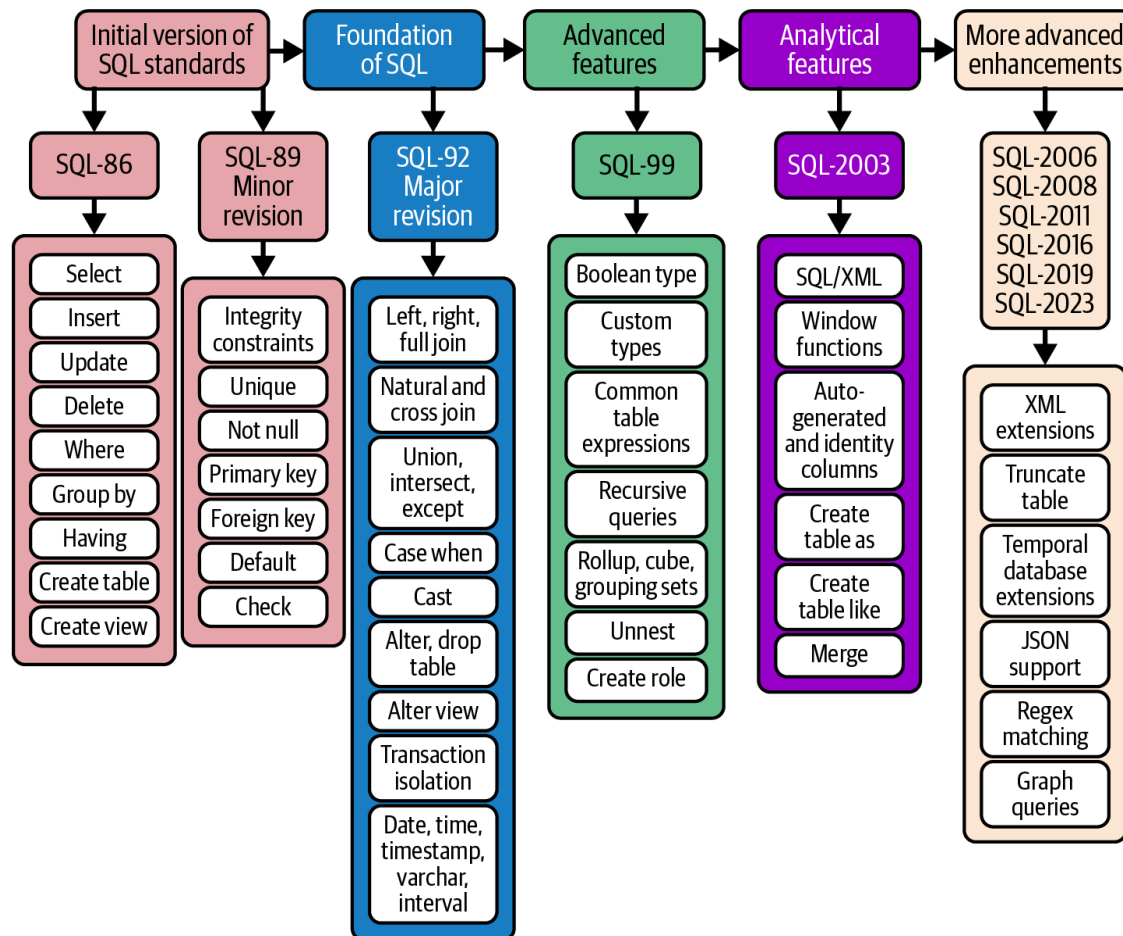


Figure 8-2. Overview of SQL standards

ACID transactions

By design, relational databases offer the strongest ACID transactional guarantee among all DSSs.

Atomicity is guaranteed by executing each statement or block of statements within a single transaction. A transaction either succeeds completely or is aborted without leaving partial or incomplete results. This is possible thanks to a transaction log mechanism known as *Write-Ahead-Logging* (WAL). Before a transaction is executed, it is logged into the WAL log. The database system may utilize the WAL to roll back changes and return the database to its initial state if a transaction fails or is interrupted. Otherwise, the transaction is marked as committed.

Consistency is primarily achieved via the use of constraints such as uniqueness, non-null values, primary key, foreign keys, and checks. Foreign key constraints are a distinguished feature in SQL used to ensure *referential integrity*. This integrity ensures that references are valid, i.e., if the presence of one value in table A requires the presence of a specific

referenced value in table B, then the value in table B must exist. For example, within a customer order table, a foreign key constraint would prevent recording an order for a product referenced from another table (such as the products table) if that product does not exist in the referenced products table.

Consistency is also guaranteed via concurrency control mechanisms that handle the concurrent execution of transactions. Among the most popular concurrency control mechanisms is *Multi-Version Concurrency Control* (MVCC), a snapshot-based mechanism that allows each transaction to see a snapshot of the data (data version) at the time of the transaction initiation, regardless of what changes take place later. This way, transactions are protected against data inconsistencies that might emerge from the actions of other concurrent transactions.

Isolation is guaranteed via mechanisms such as locking, snapshot isolation (e.g., MVCC), and isolation levels. Database locks are a mechanism that allows a given transaction to place a flag on a database object (e.g., table or row) to prevent others from concurrently performing specific actions on the same object. Locking is termed *implicit* when performed automatically by the database and *explicit* when intentionally initiated by the user. Furthermore, locking is called *pessimistic* if the lock is placed while changing a DB object and *optimistic* if placed only when the changes are being committed to the DB.

WARNING

Explicit locking may be complex and may affect your application's reliability. Key concerns in this area include lock contention, suspension, timeouts, and deadlocks. Deadlocks, where two transactions are waiting on one another to release locks, are particularly common. Additionally, locks differ in terms of the set of lock modes with which they conflict; therefore, make sure you understand the side effects of using a specific lock.

Isolation levels are used to control the visibility of concurrent transactions. The SQL standard defines four isolation levels: *Read Uncommitted*, *Read Committed*, *Repeatable Read*, and *Serializable*. These levels are de-

defined in terms of four phenomena: *dirty read*, *nonrepeatable read*, *phantom read*, and *serialization anomaly*. As this topic is a bit long to illustrate in detail, I refer the reader to the excellent documentation page of PostgreSQL on [transaction isolation](#).

Finally, durability is ensured via mechanisms such as WAL. By first logging transaction steps into the WAL log, there is a guarantee that once a transaction is committed, its results will not be lost even in the event of subsequent failures.

Analytical querying

One of the strongest arguments in favor of using SQL databases is their advanced querying capabilities. This includes table joins, aggregations, window functions, common table expressions, subqueries, stored procedures, functions, views, and many more. One major advantage of such querying features is the flexibility they provide in terms of data modeling. This is because you don't need to guess all your queries in advance before designing and creating an SQL database. Other types of DSM, such as the document model, require queries to be defined in advance in order to obtain good performance.

Schema enforcement

Relational databases require and enforce a data schema, which defines table structure, column names, data types, constraints, and more. This is an essential feature if data needs to adhere to a predefined structure. Schema enforcement improves data quality and integrity by ensuring consistency in how data is stored and accessed.

Keep in mind, however, that in many real-world applications, schemas can change quite often due to changing business needs. This is an important factor to consider when assessing the suitability of the relational model to your business problem.

With later revisions of the SQL standard, new data types such as [JSON](#) were added, which introduced some sort of schema flexibility into SQL

databases. By creating a column of type JSON, an SQL database allows users to store data with variable structures, just like a JSON file.

Data modeling with relational databases

Data modeling is predominantly used in SQL databases. This is because of their standardized design principles and features, the flexibility they offer in terms of table organization, and the business and technical intuitiveness of the relational DSM.

As we discussed earlier in this chapter, data modeling is a technique used to define and assess the data requirements necessary to support various business operations within the corresponding data infrastructure of financial institutions. That said, data modeling needs to always start with a business discussion (conceptual phase) that you later translate into an actual database design (logical and physical phases). We can discuss a lot about how to define and fulfill each of these three phases. However, to keep things simple and within scope, I will limit my treatment of SQL data modeling to the following techniques: normalization, constraints, and indexing. For each technique, I will briefly illustrate a few business use cases.

Normalization

Normalization is the most popular SQL data modeling technique. It defines a table structure following several so-called *normal forms*.

Normalization ensures that the following conditions are met:

- Data redundancy and repetition are minimized
- Data is organized in small tables instead of one large table
- Tables have well-defined relationships and references
- Updating or adding data will be done in as few places as possible
- Ability to add and remove data without altering or refactoring the tables
- Tables are neutral to queries (i.e., your table design is not constrained by the queries you want to perform)
- Tables offer clear and informative data for business users
- You avoid the problem of insertion, update, and delete anomalies

There are several types of normal forms; however, most databases are normalized using the following three forms:¹⁶

First normal form (1NF)

This form ensures that all columns are single valued. A field may not contain composite values or nested records. If such nested values exist, they should be expanded to multiple rows. An example is illustrated in [Figure 8-3](#). The table on the left is not in 1NF as it contains a multivalued column (transaction_id), while the table on the right is in 1NF as it contains no multivalued records.



user_id	account_number	transaction_id
abc123	19742987	11
vnj659	53747927	22, 33, 50

user_id	account_number	transaction_id
abc123	19742987	11
vnj659	53747927	22
vnj659	53747927	33
vnj659	53747927	50

Figure 8-3. First normal form conversion

Second normal form (2NF)

This form eliminates *partial dependency*. If a table has a composite primary key with two or more columns, then all the columns (not included in the primary key) must depend on the entire composite primary key and not on a part of it. A table exhibits partial dependency when a nonprimary key column depends on only a portion of the primary key. [Figure 8-4](#) shows an example. The right table has a composite primary key including account_id and account_type_id. In this case, the column account_type_description depends only on account_type_id, hence generating a partial dependency. To normalize in 2NF, the account_type_id should be stored in a separate table, as illustrated on the right side of the figure.

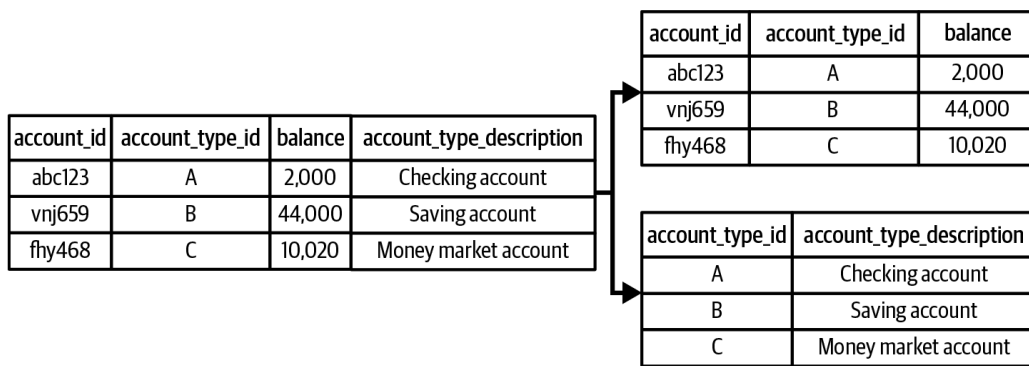


Figure 8-4. Second normal form conversion

Third normal form (3NF)

This form eliminates *transitive dependency*. All columns should only depend on the primary key, not on any nonprimary key columns. If a nonprimary key column depends on another nonprimary key column, the table is said to have a transitive dependency. See [Figure 8-5](#) for an example. The table on the left side of the figure has a single-column primary key defined for `transaction_id`. Both `account_id` and `amount` depend on `transaction_id`, but `max_transaction_limit` depends on `account_id`, which is not a primary key. This generates transitive dependency. To normalize in 3NF, we need to create a separate table that stores the `max_transaction_limit`, as illustrated on the right side of the figure.

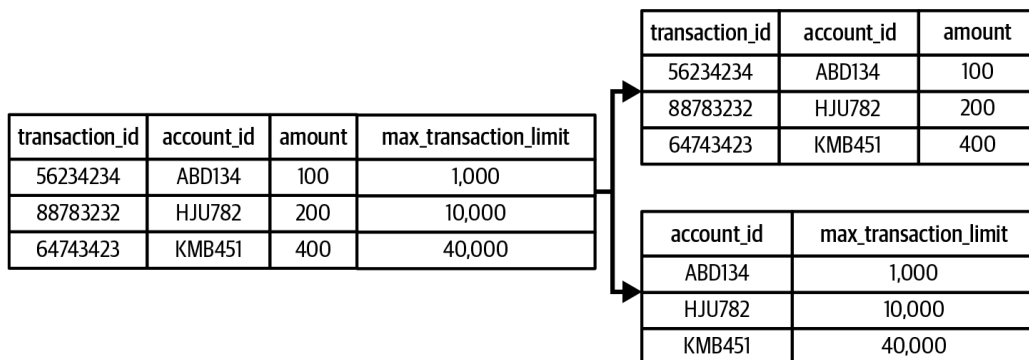


Figure 8-5. Third normal form conversion

Relational database systems often prioritize normalization as their main modeling strategy. By eliminating redundancy, normalization ensures data integrity by minimizing the number of places that store the same piece of data and establishing reliable relationships between the tables. Think of this from a compliance perspective. For example, if a client of a financial institution requests that their personal data be updated or erased, the institution will only need to check one or a few tables (e.g., a client table) to find all of the individual's information. If data was not nor-

malized, the same personal information may be duplicated in many places, making it difficult to find, aggregate, update, or delete.

Constraints

Constraints are a crucial feature of relational databases. They enforce data integrity and consistency during insert, update, and delete operations. The main advantage of defining constraints at the database level is that it decouples the constraint checks and management from the application that interacts with and manipulates the database.

Examples of SQL constraints include the following:

- The not-null constraint ensures that values in a given column are never null.
- The uniqueness constraint ensures that values in a given column are unique.
- The check constraint ensures that values in a given column satisfy a given value condition (e.g., ≥ 0).
- The primary key constraint ensures that values in one or more columns can be used as the primary unique identifier in the table.
- The foreign key constraint ensures that values in one or more columns in one table match the values in a given column in another table.

For a comprehensive overview of these constraints and how to create them, I recommend the official PostgreSQL [documentation on constraints](#). Keep in mind that constraints may have an impact on performance as they add additional work for the DB to do. The best way to approach DB constraints is by mapping them to business constraints during data modeling. For example, a check constraint may be added to ensure that the account balance never goes below zero, while a foreign key constraint may be added to guarantee that a product is not sold if it's not in the inventory, and so on.

Indexing of relational databases

If you are going to work with relational databases, indexing is a must-have strategy. SQL databases physically organize data on disk in *data files*,

with each row called a *record* and each column within a row called a *field*. To search for a specific record, an SQL database can perform a full scan of all data files. However, this is an expensive operation, especially when the data size gets very large. This is where indexing is needed.

An index is a specialized data structure stored separately from data files in index files. It is optimized to enable rapid search and retrieval of specific data records. Indexing is typically applied to the column(s) that you will use in your filtering statement. For example, suppose you are tasked with the creation of a bank transactions table that stores the `transaction_id`, `user_id`, transaction type, `transaction_time`, and `transaction_amount`. Your business team tells you that they want to query this transaction table by filtering on the `user_id`. Without an index, a query will scan the entire transaction table, which may be extremely slow and costly. To overcome this issue, you can add an index on the `user_id` column. If the business team tells you that they want instead to query `user_id` for a given time interval, then consider adding a *composite* index on both the `user_id` and `transaction_time` columns.

Relational databases offer a variety of indexes, and you can add as many indexes as you want.¹⁷ However, it is important to keep in mind that indexes get regularly updated when you add, modify, or delete records. If such operations are quite frequent, then index updates might impact performance. Additionally, be careful how you define composite indexes. For example, if you create a composite index on columns A, B, and C, then a query that filters by B only, or C only, or B and C, is likely to not benefit from the index. Furthermore, if you create an index on a column of one type (e.g., integer) and then cast the column to another type during a query (e.g., string), then the index is likely to be useless. A recommended practice is to monitor index usage to see if it's being utilized by the DSS.

Technological implementations of relational databases

A large number of database technologies have been developed following the relational DSM. These include commercial solutions such as Oracle Database, Microsoft SQL Server, IBM Db2, IBM Informix, and MySQL

Enterprise Edition as well as open source alternatives such as PostgreSQL, MySQL, and MariaDB.

A full account of the differences between all these technologies is beyond the scope of this book. A general criterion that I recommend following is the degree of compliance with the SQL standard. Some database technologies, such as PostgreSQL, are well-known for their high compliance with the SQL standard. Others, such as MySQL, may exclude some aspects of the standard in return for reliability or speed, as well as include features and extensions that are not necessarily part of the standard.

The database's technological specifications are another distinguishing criterion. One SQL database system may have a functionality that others may not have. These specifications include the following:¹⁸

- The operating system that the database system can run on (e.g., Linux, Windows, etc.)
- ACID properties
- Concurrency control and locking mechanisms
- Data size limits (e.g., max table size, max row size, etc.)
- Supported data types (e.g., varchar, integer, numeric, JSON, etc.)
- Supported constraints (e.g., foreign keys, etc.)
- Data connectors
- Security, roles, access control, and encryption
- Data replication and partitioning
- Backup and recovery
- Support for materialized views
- Supported index types (other than basic B-tree indexes)

An essential factor that might impact the choice of your relational database technology is scalability. Traditionally, relational databases have supported vertical scaling, where a small machine is replaced with a larger one with more RAM, CPU, and storage. This strategy has several advantages, such as simplicity and data consistency and integrity. However, it also has some drawbacks. First, it creates a single point of failure as it involves a single machine; second, it can easily hit capacity and load balancing limits if the workload grows quickly or unexpectedly; third, there

are physical limitations in terms of how much CPU and RAM a single machine can have. This is where horizontal scaling comes into play.

In horizontal scaling, the database system relies on a system of connected machines/nodes to distribute the load. If the workload grows, additional nodes can be added to handle the increased demand. A typical approach for achieving SQL horizontal scaling is through *read replicas*, which are additional copies of the primary database that are regularly synchronized with it, either synchronously or asynchronously. These replicas are often dedicated to handling read operations (such as SELECT queries), while the primary database continues to handle write operations (such as INSERT, UPDATE, DELETE). It is also possible for some systems to have read/write replicas.

Another horizontal scaling strategy is *sharding*, which involves partitioning the database into chunks (shards) based on a shard key (e.g., date range) and distributing them across multiple machines. This way, the traffic load gets distributed among the shards. Importantly, sharding-based horizontal scalability can be difficult to manage.¹⁹ For example, a significant amount of maintenance effort may be needed to ensure the database's integrity, involving operations such as data resharding, rebalancing, and partitioning. Furthermore, it may require incorporating complicated logic into your application's read/write methods to route requests to the relevant shard.

To overcome this issue, a new generation of horizontally scalable SQL databases emerged: *distributed SQL databases*. These databases offer native built-in scalability without the need to manually manage shards. Examples include Google Spanner, YugabyteDB, and CockroachDB. If your scalability needs are remarkably large, then such databases may be the ideal solution. For example, Deutsche Bank leveraged Google Spanner to achieve *one of the biggest IT migrations* in the history of the European banking industry, involving 19 million Postbank (Deutsche Bank's retail branch) product contracts alongside the data of 12 million customers.

Financial use cases of relational databases

Relational databases are extensively used within the financial sector. This can be explained by a number of factors. First, a significant part of financial data is tabular time series and panel data, which is perfectly suited for the relational DSM.

Second, financial professionals perform a lot of analysis related to business intelligence, forecasting, pricing, risk management, and modeling. Relational databases are an ideal solution for these tasks due to their powerful and intuitive analytical querying capabilities.

Third, relational databases offer a lot of the reliability features that most financial applications require, such as data consistency, integrity, and a transactional guarantee. For example, financial payment applications require transactions to be idempotent, meaning if they are executed multiple times, the outcome should be consistent. If you purchase an item for \$100 and the application processes your payment twice, idempotency ensures your account is debited only once for \$100. SQL databases can achieve idempotency through mechanisms like uniqueness constraints and idempotency tokens. For instance, if each payment is assigned a unique idempotency token and the column storing these tokens has a uniqueness constraint, any attempt to insert the same payment twice will be rejected by the database due to the violation of the uniqueness constraint.

Payments are one of the most essential operations in financial markets. A large number of players are involved in payment processing, including SaaS applications, online shops, ecommerce websites, retailers, payment gateways, card networks, payment processors, and payment service providers. Crucially, designing a payment system can be quite challenging, as it needs to ensure the following:

- Scalability (imagine how easily the volume of payments can increase)
- Data durability (people won't like it if they can't find their historical payment records)
- High availability and zero downtime (it's not a pleasant experience if you can't pay)
- Data consistency and correctness (people won't tolerate mistakes with their money)

How do you build a payment system that meets all the above requirements? First, let's try to understand how payments are processed.

According to [Stripe](#), a typical card payment transaction involves the following steps:

1. The customer initiates a payment by providing their payment details (e.g., credit card) at the business payment channel (e.g., online shop).
2. A payment gateway receives the transmitted information, encrypts it, and passes it to the payment processor.
3. The payment processor forwards the information to the acquiring bank (the online shop's bank), which in turn forwards the information to the issuing bank (the customer bank) through the relevant card network (e.g., Visa or Mastercard).
4. The issuing bank verifies the request and responds with an approval or rejection message sent back to the payment processor via the same path (card network and acquiring bank).
5. The payment processor communicates the transaction outcome to the business, which proceeds to conclude the purchase or inform the customer of any issue.
6. If the transaction is concluded, a clearing and settlement process takes place in which the transaction amount is transferred from the issuing

bank into the acquiring bank, which in turn deposits the funds into the business's account.

Importantly, when data passes through this chain of information exchange, some of the involved entities store a portion of the payment data for reasons such as transaction reviewing, reporting, reconciliation, and fraud detection. Due to the mission-critical nature of payments, a highly available, scalable, and reliable database system is required. This is where distributed SQL databases can excel.

A prominent solution in this market is CockroachDB, a cloud native, distributed database based on standard SQL and developed by [Cockroach Labs](#). CockroachDB offers a number of valuable features for mission-critical applications:

- Simple horizontal scalability where users can add additional nodes as needed. Interestingly, even though it's a distributed system, CockroachDB works as a single logical database, enabling data access from any node, regardless of its location.
- Support for transaction-heavy workloads with [distributed atomic transactions](#).
- High availability with no downtime, achieved via a [consensus algorithm](#).
- Multiactive availability (each node can serve both read and write requests).
- Support for multiregion and multi-cloud (ideal for compliance that restricts data residency to a specific region).

To give an example of the application of CockroachDB in the financial sector, let's take the case of [Shipt](#), an American delivery service owned by Target Corporation. As an online ecommerce company, Shipt payment services are core to its business model. The Shipt team was tasked with the creation of a distributed database system that meets the requirements of a reliable payment system, in line with what we discussed earlier in this section. In particular, Shipt wanted a multiregion payment service that could ensure [correctness](#) throughout the entire payment cycle.

By leveraging CockroachDB, the Shipt engineering team managed to build a reliable, correct, cloud native, multiregion, and highly available payment data management system. Regional replication allowed Shipt to achieve lower transaction latency. Furthermore, to ensure idempotency throughout the payment lifecycle, Shipt relied on idempotency [tokens](#). Overall, by building its system on top of CockroachDB, Shipt can now support its business growth across different regions and markets with a resilient and scalable payment architecture.

The Document Model

A highly reputable DSM is the document model, which stores information in a document format such as JSON or XML. This section will specifically focus on JSON-based DSMs, which are the predominant choice in the industry. A document looks like the following:

```
{
  "document_id": 1,
  "legal_name": "Microsoft Corporation",
  "type": "public company",
  "isin": "US5949181045",
  "symbol": "MSFT",
  "sector": "Information technology",
  "products": [
    "Software development",
    "Computer hardware",
    "Social networking",
    "Cloud computing",
    "Video games"
  ]
}
```

A DSS designed to store and query documents is called a document-oriented database, or simply a document database.

Why document databases?

Document databases are extensively used for all kinds of purposes, powering some of the world's largest applications. Among their most desired

features are the following:

Schema flexibility

A document database can store any document, regardless of its content structure. In other words, document databases do not enforce schemas natively, unlike SQL databases. This allows businesses to quickly develop an application and easily change its logic. You can add, change, or rename the document fields based on your business requirements. Make sure, however, to be aware of the potential side effects of such flexibility; if not managed properly, it might impact data integrity and consistency. At some point, you might even want to enforce a schema in the documents. This is often achieved via a schema validation mechanism implemented on the application side.

Document modeling

Document formats such as JSON are quite familiar and might be easier to work with. Moreover, unlike relational or other types of data storage formats, documents map directly to objects such as hash tables or classes in most popular programming languages, without the need to add an *Object Relational Mapping* (ORM) layer to your application.

Horizontal scalability

Document databases are distributed by design, which means that they can scale horizontally. This makes them an ideal choice for modern data-intensive applications. Moreover, being distributed, document databases provide resiliency and availability through multinode data replication.

ACID (atomic) transactions

Document databases support ACID transactions, at least for single-document transactions, and in many cases for multidocument transactions as well. Crucially, it is often a less strict version of ACID that focuses on achieving data consistency via atomicity and an isolation guarantee (e.g., using snapshot isolation).

Document databases are quite performant when it comes to high-volume data reads/writes. This can be partly explained by the distributed architecture of document databases, which splits the load among the many nodes that make up the system. Furthermore, unlike traditional SQL databases, document databases do not necessarily enforce schema checks, standards, constraints, and ACID properties, which in turn increases query performance. Keep in mind that this is not a shortcoming, but rather a design choice.

Data modeling with document databases

In document databases, data modeling must be seen from a somewhat different viewpoint than that of the SQL DSM. Technically speaking, document databases are not designed for performing complex queries and table joins as is customary with SQL databases. This means that the range of queries that you can perform in a document database will be constrained by your data model. For this reason, the first step in document data modeling needs to be a business discussion to think and define in advance the queries and filters that users will want to perform. This is often called *query-driven data modeling*. User-defined queries should serve as the foundation for the subsequent data modeling stages. In this section, I will discuss three document data modeling techniques: document and collection structure, denormalization, and indexing.

Document and collection structure

The central concept of a document database is the *document*. You can think of documents as the equivalent of rows in the relational model. A document stores data objects as a set of key-value pairs. It can store many types of data, including strings, integers, dates, Booleans, arrays, and even subdocuments. Documents are then organized in *collections*, which are the equivalent of tables in the relational model.

Unlike SQL databases, there are no standardized rules for modeling a document database. In this case, the modeling process is mostly driven by business needs. As far as collections are concerned, the main thing to con-

sider is that document databases do not allow for performing joins; therefore, collections should be modeled in a way that minimizes inter-collection relationships and at the same time maximizes intra-collection data cohesion. For example, a separate collection may be created for each business entity, such as users, products, transactions, subscriptions, reference data, prices, and quotes.

NOTE

Managing schema changes in a NoSQL database can be challenging, especially in environments where the schema is not strictly enforced and evolves over time. A number of good practices can be adopted to mitigate this issue. One such practice is schema versioning, which can be implemented by defining and storing your document schema in a separate location, and adding a `schema_version` field to your documents. Each version of the schema should have a unique identifier. Furthermore, you can implement schema validation on the application side to check if your document matches the referenced schema. Make sure you document and communicate schema changes clearly within the team.

Denormalization

To ensure intra-collection cohesion, all related data needs to be stored in the same document. This strategy is often called *denormalization*, as it involves making redundant copies of the data in multiple collections to increase performance and avoid data joins. As an alternative, it is possible to use *references* that link documents across collections. You can think of references as a soft version of foreign keys in SQL databases. For example, a user ID field in the *transactions* collection can be used to find a related document in the *contacts* collection.

Indexing of document databases

Document databases are designed for large amounts of data. As such, indexes are often used to improve query performance. An index stores a small portion of the collection data in an easy-to-traverse data structure. Without appropriate indexing, a document database will have to scan the entire collection when searching for a specific document.

The two primary types of indexes often supported by document databases are *single-field* indexes and *composite* indexes. Deciding which and how many indexes to create depends on your queries and it is crucial for improving the performance of your application. If a large number of your queries filter by a single field only, then it's better to create a single-field index for each of these fields. If you also have queries that repeatedly filter by multiple fields, then you can create a composite index on those fields. Other types of advanced document indexes exist. For example, multikey indexes are used to index array fields, while text indexes are used to support text search queries on string content.^{[20](#)}

Importantly, there are a few factors to consider while indexing a document database. First, although indexes improve read performance, they will negatively impact write operations as they must also update the index. This is especially the case for applications with a high write-to-read ratio. Second, several document databases impose a limit on the number of indexes that a collection can have. If you run out of indexes, then you might not be able to meet your business or performance requirements anymore. Third, indexing might become more challenging if you have a highly nested or complex document structure. For this reason, consider making your documents as flat as possible to achieve optimal indexing results.

Technological implementations of document databases

There are a large number of data storage technologies that implement and support the document model. The most prominent examples include open source options such as MongoDB and Apache CouchDB, as well as managed commercial solutions such as Amazon DynamoDB, Azure Cosmos DB, and Google Cloud Firestore. Cloud-based solutions such as DynamoDB, Google Firestore, and Azure Cosmos DB are commonly preferred as they offer tight integration with other cloud services as well as reduced infrastructure and security overheads.

Moreover, a few specialized document-oriented data storage options are available. The most prominent example is *Elasticsearch*, which can be described as a distributed search and analytics engine. It is built on the

Apache Lucene search software, which implements a well-known technique called inverted indexing. Elasticsearch is often used as a secondary DSS, where data from one or more primary DSSs is regularly pulled and indexed to allow easy and fast searching.

Financial use cases of document databases

Document databases have received considerable attention within the financial industry. This is particularly the case for financial applications that require scalability, latency, availability, schema flexibility, and integration capabilities, which are characteristic of document databases.

Let's take MongoDB, for example. Financial institutions have leveraged MongoDB for various business applications. A common example is the [development of payment systems](#). MongoDB's flexible data schema enables payment applications to accept and enrich any payment data structure and type. In addition, MongoDB may offer the necessary scalability and availability features that are essential to payment systems. Moreover, MongoDB's API-based data management is an ideal fit for the payment business, which relies significantly on APIs.

Wells Fargo is a well-known financial services corporation with a remarkable global reach. To modernize its credit card payment system, Wells Fargo launched the *Cards 2.0* initiative. Based on an industry [case study](#), the main goals of the initiative were the following:

- Ensuring a seamless multichannel experience for credit card customers (branch, mobile, digital, etc.)
- Reusable and scalable data APIs to enable the quick rollout of changes across multiple channels
- Reduced dependency on third-party card processors

To achieve success with Cards 2.0, Wells Fargo had to reduce its reliance on mainframe infrastructures. The reason, according to the case study, is that “while mainframes hold critical System of Record (SOR) data, they often bring technical debt, dependencies, and are increasingly costly to manage—not least because there’s a shortage of people with the right skills to maintain them.”

While the bank is responsible for issuing the cards, the gateway that tracks all the transactions is the Fiserv mainframe system. Relying on data ingestion mechanisms based on a mainframe was not ideal for modern multichannel applications. To solve this problem, the engineering team at Wells Fargo designed a modern data infrastructure powered by MongoDB. The new solution included batch (Apache Spark) and real-time (Apache Kafka) systems that listen and pull the data from the mainframe and upload it to MongoDB. The resulting MongoDB-based DSS was an Operational Data Store (ODS) that could serve data to many business channels.

The Wells Fargo team didn’t expose MongoDB collections directly to data consumers. Instead, data APIs were designed to serve various types of data (e.g., accounts, transactions, etc.). Such data APIs are then imported and used in the various microservices. Using MongoDB, Wells Fargo was able to handle over 20+ terabytes of daily data and move from a monolith architecture to a modular architecture of more than 80 microservices.

Another noteworthy example of a reliable and scalable document database technology for financial services is AWS DynamoDB. DynamoDB is a key-value and document-oriented database designed for high speed, throughput, availability, scalability, and millisecond latency. To illustrate its capabilities, on 2022 Amazon Prime Day, Amazon DynamoDB reliably [handled 105.2 million requests per second](#) across trillions of API calls with single-digit millisecond performance. In addition, DynamoDB can be natively integrated with AWS Lambda to execute a data processing logic each time a DynamoDB table is updated. For example, the Amazon finance technologies (FinTech) payment transmission team, which supports and manages products for the accounts payable (AP) team, [implemented a similar architecture](#) to ensure the scalability and timely processing of remittances at Amazon.

The Time Series Model

In [Chapter 2](#), we talked about time series data and illustrated its main characteristics. In essence, a time series is a sequence of measurements or observations of one or more variables tracked at increments in time. Examples include temperature, resource consumption by a computer, events, clicks, and stock prices.

Financial time series data such as stock prices are now generated in large volumes and velocities and are widely employed to analyze temporal market dynamics. This includes the analysis of historical trends, averages, variances, anomalies, correlations, stationarity, and cycles.

Moreover, as financial time series data has increased in volume, variety, and velocity, its range of application has significantly expanded. Time series analysis is now integral to financial analysis and forecasting and can be used in predicting stock prices, market risks, interest rates, foreign exchange fluctuations, and many more.

As a result, demand has emerged for a new type of a DSM to enable efficient storing, manipulating, and querying of time series data. I shall refer to this model as the *time series DSM*. A notable increase in the popularity of time series databases has been [observed in recent years](#). To under-

stand why, the next section will highlight the key characteristics that distinguish time series databases.

Why time series databases?

A time series database is a specialized type of DSS designed specifically to implement the time series DSM. Time series databases offer several advantages, such as the following:

- A specialized engine designed for storing and processing time series data, providing substantial performance enhancements tailored for applications that handle large volumes of time series data, such as trading systems.
- Built-in functionalities to perform efficient time-based aggregations, such as temporal grouping of data (yearly, monthly, daily,..) and temporal transformations (e.g., simple moving average, exponential moving average, cumulative sum, percentile).
- Fast queries enabled through optimized time series indexes and in-memory caching.
- Simple data model based on the time association of entities.
- Data immutability: time series databases are often designed for immutability and append-only behavior—i.e., once recorded, data is rarely updated.
- Efficient data lifecycle management by keeping recent data in memory, and deleting or compressing old data for efficient storage.

Keep in mind that the time series DSM is a specialized model; therefore, it should be considered only when your business and application requirements are centered around the efficient storage and processing of time series data. If the time series dimension of your application is just one of many aspects, then you might want to go for a more general-purpose DSM such as the SQL or document model.

Data modeling with time series

Data modeling in the time series DSM is as simple as the structure of time series data: a timestamp and associated data. The equivalent of a table in a time series database is often called a *measurement*. A measurement is a

container of at least three main elements: time, fields, and metadata.

[Table 8-1](#) illustrates an example.

Table 8-1. A measurement in a time series database

Time	Price	Ticker	Exchange
2019-02-18T16:12:01Z	15.45	ABC	NYSE
2019-02-18T16:13:01Z	15.41	ABC	NYSE
2019-02-18T16:20:01Z	15.44	ABC	NYSE
2019-03-06T17:33:20Z	345.45	ZYK	NYSE
2019-03-06T17:33:25Z	345.47	ZYK	NYSE

In [Table 8-1](#), the first column is time, which in a time series database must always be present. The next column, price, is a field. Field values represent the actual data in a measurement, and each measurement should have at least one field. In our example, we are observing the price of stocks over time. The final two columns, ticker and exchange, are referred to as metadata or tags. Tags are optional, although they are quite useful. Most time series databases construct indexes based on tags rather than time or field columns. A single row in a measurement is called a *point*. A set of points that share the same set of tag values is called a *series*. For example, the first three rows in the measurement shown in [Table 8-1](#) form a series, since they hold data related to the price of the ticker ABC on the NYSE exchange.

Technological implementations of time series databases

A variety of time series database implementations have been developed in the market. These can be divided into native and non-native implementations.

A non-native implementation is not primarily designed for time series data, but it can be used for this purpose. For example, it is possible to use an SQL database such as PostgreSQL to create a time series table like the one shown in [Table 8-1](#). To boost performance, an index like a B-tree or

BRIN (Block Range Index) can be created on one or more tag columns plus the timestamp column. Furthermore, PostgreSQL supports index-based table clustering, which can enhance the efficiency of queries that filter by a given temporal interval. Similarly, a document database such as MongoDB can be used to store a time series as a set of documents in a collection indexed by time and tag keys.

Crucially, while SQL and document databases may be able to handle time series data, they are not optimized for this purpose. Two options are available to overcome this issue. First is time series extensions created on top of general-purpose databases. A prominent example is TimescaleDB, which extends PostgreSQL with time series characteristics. TimescaleDB is quite popular due to its performance, scalability, and compatibility with PostgreSQL and the SQL query language. As another example, MongoDB introduced [time series collections](#), which allow for optimized storage and retrieval of time series data. To do this, MongoDB time series collections use a columnar storage format and store data in time order. Moreover, upon the creation of a time series collection, MongoDB automatically creates a composite index on the time and metadata fields.

The second and most reliable option for handling time series data is native purpose-built time series databases. A good example is kdb+, developed and maintained by KX. Kdb+ is designed with high performance and scalability features in mind. It stores the data in a columnar format, allowing for efficient data compression and fast queries. In addition, it uses an in-memory compute engine for fast processing and querying in real time. Furthermore, kdb+ supports the q language, which is known for its powerful and efficient querying capabilities.

Some database systems may frequently be referred to as in-memory databases or as having in-memory characteristics. What this means is that the database system keeps all or part of its data in memory (i.e., RAM). By storing data in memory, the database achieves very low response times (i.e., low latency) by eliminating the need to fetch the data from disk. It is well-known empirically that RAM-based access is substantially faster than disk access, precisely random disk access. In-memory data access capabilities can be crucial for time-critical applications such as trading and online stores. A common data management strategy with in-memory databases involves storing the most current data in memory while keeping historical data in a separate location.

There are different types of in-memory databases. On the one hand, there are native in-memory databases such as Redis and kdb+. Among financial institutions, the [Oracle TimesTen in-memory database](#) has been widely used. Alternatively, there are in-memory extensions designed to improve the performance of existing DSSs. For example, the Amazon DynamoDB Accelerator was created to add in-memory capabilities to DynamoDB tables. To boost the performance of their primary database product, Db2, IBM introduced BLU Acceleration, which adds in-memory processing capabilities and columnar storage to enhance the speed of data analytics. Open source in-memory storage and computing frameworks such as Apache Ignite also exist.

Two things are worth noting when working with in-memory databases. First, they are not to be treated as a primary DSS as data in memory is considered volatile (if the machine is shut down, the data will be lost). Second, data stored in memory may become invalid and require a refresh. Such a practice is called [cache invalidation](#) and may be a challenging issue if not understood properly.

Another popular time series database is InfluxDB, developed and marketed by InfluxData. InfluxDB is a native time series database designed for high performance and intuitiveness. InfluxData uses *InfluxQL*, an SQL-like query language for querying data in InfluxDB databases.

InfluxDB implements a data model that organizes data around buckets, databases, retention policies, series, measurements, tag keys, tag values, and field keys. InfluxDB's engine relies on an optimized data storage format called a Time-Structured Merge Tree (TSM) and an indexing technique called Time Series Index (TSI).²¹

Financial use cases of time series databases

Time series databases were originally developed for financial applications. As new market structures such as high-frequency and electronic trading mechanisms emerged, the volume of trades, quotes, and prices experienced a substantial increase. For example, the NYSE, which currently allows trading for all 8,000+ US securities, reports that an average of 2.4 billion shares exchange hands on a daily basis. In addition to volume, the speed of data generation has also increased. Nowadays, high-frequency trading at the NYSE occurs at the nanosecond level (i.e., one-billionth of a second). Consequently, a single second of trading can have hundreds of trades and quotes and reveal a variety of market patterns.

Multiple firms, in particular those involved in high-frequency and algorithmic trading, can leverage this fine-grained market view. However, to stay competitive, rapid data access is crucial. This is where highly performant DSSs, such as time series databases, are required.

Kdb+ has become a primary choice for financial markets, particularly in high-frequency trading, due to its exceptional performance and speed.²² One of the main challenges that firms using kdb+ face is the need for fast access and analysis of critical market data in real time.

The Message Broker Model

In distributed systems, it is common for multiple applications to need to communicate with each other to complete specific tasks. A typical scenario involves one type of application, known as *producers*, generating and storing data messages asynchronously²³ in a shared data store, and another type of application, called *consumers*, reading and processing the

messages from the same store. This communication pattern is known as the *producer-consumer pattern*. [Figure 8-6](#) provides an example.

A DSM that follows the producer-consumer pattern is called a message broker DSM, and a technology that implements the message broker DSM is simply called a *message broker*. In the next few sections, you will learn about the main features of message brokers, their data modeling principles, technological implementations, and, lastly, their use cases in the financial sector.

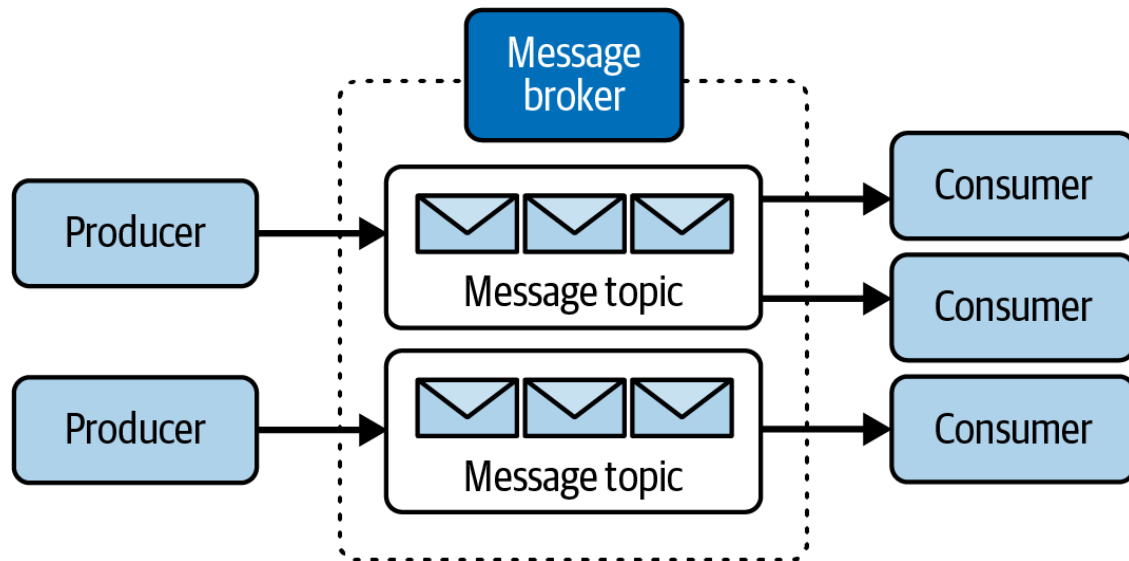


Figure 8-6. The message broker model

Why message brokers?

Message brokers are widely used in developing data-intensive systems, particularly in event-driven, distributed, and streaming architectures. In these contexts, data is generated and consumed by various applications following different patterns and scales. To meet such requirements, message brokers offer the ideal features in terms of simplicity, speed, and scalability. Importantly, a message broker is not to be used as a primary or persistent DSS but rather as an intermediary that facilitates the exchange of messages between applications.

By decoupling data production and consumption, message brokers enable producer and consumer applications to work and scale independently. If a small number of messages are being produced, a few consumers can handle the workload. However, if the number of messages increases sig-

nificantly, additional consumers can be added to handle the increased load. Moreover, producers need not worry about who the consumers of the data will be, which allows for flexibility in adding different types of consumers based on business needs.

A distinguishing feature of message brokers is their fault tolerance. Consumers and producers can be easily replaced in case of failure without impacting the state of the message broker. Moreover, message brokers are quite simple to use as applications only need to know the topic for publishing or consuming messages. Let's discuss message topics in the next section.

Data modeling with message brokers

Message broker data modeling revolves around two main concepts: topics and message schema, which I will illustrate in this section.

Topic modeling

Topics are the main building blocks of message brokers. A topic is a unique container of messages that publishers and subscribers need to specify when communicating with each other. Think of topics as the equivalent of tables in SQL databases.

Topic modeling involves defining the topics and their target consumers and producers. They can be created based on business requirements. In some cases, certain message brokers support multiple types of topics, each optimized for a specific need. For example, suppose you have an online application offering customer support for five categories of issues, each handled by a different backend system. If you decide to use a message broker, then you may want to create five topics, each handling a different type of client request. In this case, if Topic 1 handles issues of Type A, then only messages related to issue Type A will be published to that topic, and only backend systems that handle issue Type A will be able to process messages from Topic 1.

Naturally, large-scale applications can have hundreds or even thousands of topics. In this case, establishing data governance policies for topic man-

agement becomes critical. This might include the people and methods involved in creating a topic, the producer and consumer applications that are permitted to use it, the type and format of data that can be submitted to the topic, data quality checks and validations, data anonymization, encryption, and data loss management.

Message schemas

Message brokers do not enforce a schema on message structure, offering flexibility in message definition. However, if consumers don't know what information to expect from producers, things might get out of control. This situation may force applications to manage multiple formats, which adds unnecessary complexity.

Therefore, it is a good practice to define a message schema for each topic initially and then implement validators on both the producer and consumer sides. These validators ensure that the message content is checked for correctness before it is published or consumed.

The most popular format for message definition is JSON. However, it is also possible to use XML or other formats.

One of the best practices in designing message brokers is the establishment of a message schema registry. This consists of a centralized repository for storing, managing, versioning, and validating schemas for topic message data. Think of the schema registry as data governance practices that allow producers and consumers to communicate over a well-defined data contract in the form of a schema.

Apache Kafka, which is discussed in more detail later in this section, is a leading example of a message broker that [supports a schema registry](#). The schema registry in Kafka facilitates schema evolution and compatibility, enabling producers and consumers to update their schemas while preserving data integrity and cross-version compatibility. This capability is especially valuable for ensuring that data written to Kafka remains well structured and can be accurately interpreted by downstream consumers.

A technical aspect to keep in mind about message brokers is that they typically require messages to be serialized before being submitted to a topic. The message consumer then deserializes the message back into its original structure. Serialization is the process of converting a data object into a format optimized for transmission and storage, typically a byte stream. Deserialization reverses this process, converting the byte stream back into the original data object. While custom serializers and deserializers can be created, it is common to use built-in ones for formats such as JSON, Avro, and Protocol Buffers.²⁴

Technological implementations of message brokers

Several technological options are available for implementing a message broker. Examples include Apache Kafka, RabbitMQ, Redis, Google Pub/Sub, Apache ActiveMQ, Amazon SQS, Amazon SNS, and Azure Service Bus. Making a full comparison between such technologies is beyond the scope of this book. However, the following criteria can be used to perform a technical assessment:

Some message brokers are designed for specific use cases, such as message queues (e.g., AWS SQS), while others are better suited for multiconsumer notifications (e.g., AWS SNS).

Performance

Message brokers differ in throughput and message read/write latency. For instance, Apache Kafka excels in throughput, whereas Redis is known for its low latency.

Delivery mode

Message brokers offer different levels of message delivery guarantees, such as “At Most Once,” “At Least Once,” and “Exactly Once.” For more details, refer to Cloudflare’s [documentation page](#).

Message persistence

Some message brokers purge messages from their store upon delivery to consumers (e.g., Apache ActiveMQ and RabbitMQ), while others store messages in a commit log where the same message can be consumed by multiple consumers and at a later point in time.

Scalability

Message brokers can vary in their scaling capabilities. Some excel at scaling message production, enabling the publication of a large number of messages per second, while others are more effective at scaling consumption, allowing for many concurrent consumers. For example, Apache Kafka is well-known for its scalability features. One way this can be achieved is by allowing a topic to be further divided into partitions. This way, instead of having one consumer blocking a topic, partitions allow multiple consumers to read messages from the same topic by referencing the partition ID within the topic from which they want to consume data.

Message prioritization

Some message brokers, such as RabbitMQ and ActiveMQ, offer message prioritization features whereby high-priority messages are consumed before low-priority messages.

Message ordering

Message ordering refers to the order in which messages are consumed from a topic. While some message brokers may not enforce any specific ordering, others ensure that messages are consumed following a specific order, such as consuming the first messages received first.

When developing a cloud-based data infrastructure, managed cloud message brokers can be an excellent choice. These solutions are specifically engineered to seamlessly integrate with other cloud services. For example, you can configure topics to automatically receive messages when a file is uploaded to cloud storage. Then, a cloud function can consume that message from the same topic to perform a specific task related to that file.

Financial use cases of message brokers

Message brokers have found a variety of applications in the financial sector, particularly in the development of event-driven and real-time systems. They are extensively used for managing operations involving a continuous stream of high-volume data such as payments, credit card transactions, loan applications, ATMs, and mobile notifications. Message brokers can streamline tasks such as fraud analysis, application and transaction approvals or rejections, credit authorizations, and client notifications.

For example, At PayPal, [Apache Kafka has been used extensively](#) to support various critical operations such as first-party tracking, metrics streaming, database synchronization, log aggregation, batch processing, risk management, and analytics, with each of these use cases processing over 100 billion messages daily. PayPal's Kafka infrastructure includes 1,500 brokers across 85+ clusters, 20,000 topics, and nearly 2,000 Mirror Maker nodes, achieving 99.99% availability. In the 2022 Retail Friday, PayPal's Kafka processed a traffic volume of 21 million messages per second, which totaled about 1.3 trillion messages in a single day. Similarly, Cyber Monday of the same year resulted in 19 million messages per second, totaling 1.23 trillion messages in a single day.

Another significant application of message brokers in finance is the development of highly scalable and efficient applications that handle real-time financial data sharing, messaging, and streaming. For example, a financial data provider can leverage Apache Kafka to offer various types of data by organizing them into different topics. Subsequently, clients are subscribed to a specific topic based on their subscription plan. Kafka uses *Access Control Lists* (ACLs) as an authorization mechanism to determine which users are allowed to perform which actions on Kafka resources. Moreover, within the same topic, it is possible to further categorize the data into partitions (e.g., by ticker) and offer clients the option to subscribe to a subset of the partitions. Kafka has the concept of a *consumer group*, which can be used to allow different clients to consume the same data from the same topic/partition using a Kafka consumer offset. An offset is just a pointer that indicates the position within a partition of the next message to be consumed by a consumer group. ²⁵

ING Group is a Dutch multinational financial services corporation that provides a wide range of services such as retail and commercial banking, investment banking, and insurance services. ING, like other commercial banking firms, has a mechanism in place to identify fraud in its online banking activities. However, over time, this system became overly complex, expensive, unreliable, and challenging to scale effectively to manage the growing volume of data required for real-time fraud detection.

To overcome these issues, the ING engineering team decided to leverage Apache Kafka as the main *event bus* to support real-time fraud detection and other data streaming use cases. In a [presentation given by ING experts Timor Timuri and Richard Bras in 2018](#), they illustrated the high-level use case of Apache Kafka.

To handle client-sensitive data, the ING team introduced security into the messages stored in Kafka via a symmetric end-to-end encryption module. This works by first encrypting the data before publishing it to a topic and then decrypting it upon consumption by the consumer.

The Protocol Buffers (protobuf) format was chosen for message serialization/deserialization. Before publishing, messages are serialized into protobuf, and when consumers receive the message, they deserialize it.

In addition, the ING engineering team created a number of predefined client settings to allow users who just want to use Kafka and do not care about the details (e.g., publish messages to a topic). This was provided via a simple and easy-to-use interface. Furthermore, test components were developed to simulate message publication, and monitoring was introduced to evaluate Kafka's performance. For compliance purposes, all events are persisted in a Cassandra data store.

To ensure availability, ING's team adopted a multi-data center strategy to replicate data and ensure availability in case of downtimes or disasters.

Crucially, while Kafka initially served for real-time fraud detection at scale, it later expanded to become a primary development platform sup-

porting a wide array of applications. By 2018, ING managed 200 topics, processing 1 billion messages daily with peak rates of 20,000 messages per second. Kafka is utilized across over 100 teams at ING, catering to 120 distinct use cases.

The Graph Model

In [Chapter 2](#), we explored graph data, illustrating its structures, types, and financial applications. One of the main advantages of graph data is its ability to reveal complex patterns and mechanisms that simpler data types (e.g., time series) cannot uncover. As a result, a wide range of applications have been created, the primary functionality of which revolves around the storing and processing of graph data. The graph DSM is intended for such use cases. A technology that implements the graph DSM is known as a graph DSS or, more commonly, a *graph database*.

In the following sections, we will examine the key aspects of the graph DSM, graph databases, graph data modeling principles, technological implementations, and, finally, financial use cases.

Why a graph model?

The graph DSM is used when designing graph-oriented applications that prioritize the storing and analysis of data relationships. Examples of such requirements include the following:

Centrality analysis

This aims at identifying the most important nodes in a network. For instance, it can be applied to identify *Systemically Important Financial Institutions* (SIFIs), whose failure could potentially disrupt the entire financial system, or market participants who play major roles as intermediaries, facilitating various transactions.

Search and pathfinding

This aims at locating nodes with specific attributes or determining the most efficient path between two nodes within a graph. A widely

used approach is shortest path analysis, which identifies the path between nodes with the fewest edges or minimal weights.

Community detection

This aims at finding cohesive regions or subgraphs of connected nodes within a graph. Such subgraphs are called communities. Examples include stock market clusters (stocks that move together), interbank lending clusters (e.g., banks that frequently transact with each other), and fraud detection (e.g., groups of transactions that might be linked to fraud rings).

Contagion analysis

This examines how a shock impacting one or a set of nodes propagates throughout the rest of the network. Applications include financial stress testing, cascade failures, and systematic risk analysis.

Link prediction

This aims at identifying pairs of nodes that are likely to establish a link in the future (e.g., two financial institutions establishing a correspondence banking relationship).

It's important to remember that the graph data element must be crucial to your application to justify using a graph DSM. If your application needs to store graph data but does not require specialized graph capabilities, then a general-purpose model such as SQL may suffice.

For instance, consider a financial graph where nodes represent banks and links represent asset exposure (e.g., how many assets bank A holds at bank B). If bank A is primarily concerned with its direct exposure to the network, an SQL DSM can handle this efficiently. You can create a table with three columns: `source_bank`, `target_bank`, and `exposure`. To get your bank's total exposure, you simply run `SELECT target_bank, exposure WHERE source_bank = "YOUR_BANK_NAME"`. However, if you need to calculate the exposure of a bank to which your bank is exposed (similar to finding the friends of your friend), the SQL model would require complex joins and recursive operations to achieve this. This is where the graph

DSM excels, as its storage and processing logic is specifically designed for such tasks.

Data modeling with graph databases

Graph data modeling is quite intuitive and flexible. It involves defining a domain that consists of nodes and links, each with their own attributes and labels. Graph data modeling is often called whiteboard friendly, as it can be visually drawn on a whiteboard.

Graph data modeling involves two primary aspects: node modeling and link modeling. In node modeling, you define the different categories of nodes that can exist in the graph, such as persons, organizations, countries, or assets. Each node category is characterized by specific attributes that provide descriptive information, including category labels and additional fields for unique identification.

In link modeling, the focus shifts to defining the relationships or connections between different node categories. This involves specifying the types of links that can exist and assigning link attributes that describe the characteristics of these connections. Figure 8-7 provides an example of graph data modeling.

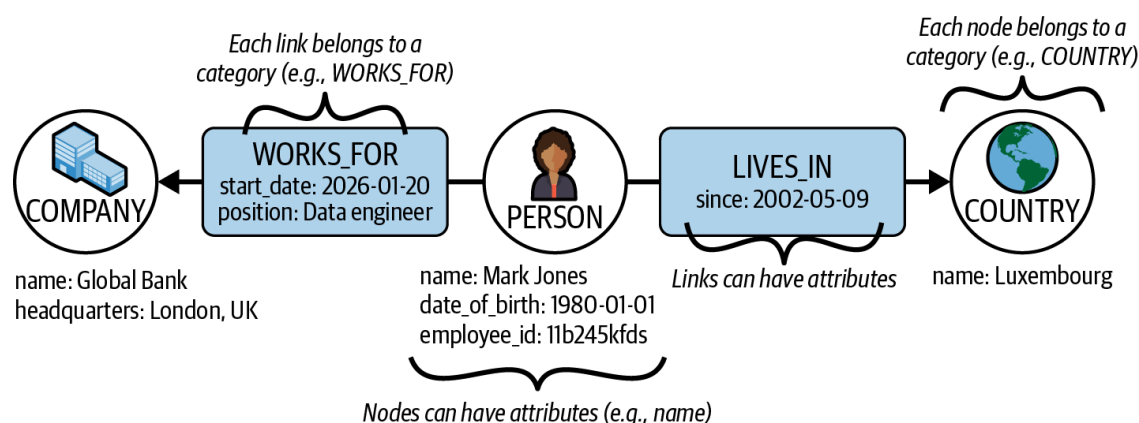


Figure 8-7. An example graph data model

Keep in mind that the graph DSM does not enforce a rigid schema, as is the case with the SQL model. New types of nodes and links can be easily added by defining their label and attributes, and existing nodes and links can be adapted to reflect new business requirements.

To enhance performance, graph data modeling may include the definition of indexes on nodes and links. These indexes are typically based on attributes associated with nodes or links.

Technological implementations of graph databases

The graph DSM can be implemented using a variety of technological options. The simplest but least performant approach involves the use of multi-model non-native DSSs such as relational databases. In this case, nodes, links, and their attributes are defined in normalized tables, and indexes can be added to improve search performance. If you recall the section on graph data in [Chapter 2](#), this would be similar to the *edge list* graph representation. This solution is recommended if your application doesn't prioritize graph data processing or if you want to perform basic graph queries.

Some relational databases have ad hoc extensions that provide graph database functionalities. The best example is [Apache AGE](#), a PostgreSQL extension developed to provide graph processing and analytics capabilities. AGE provides users with the capability to compose graph queries using a hybrid query language that integrates SQL with openCypher, an open source implementation of the Cypher graph query language originally developed by Neo4j.

Non-native graph implementations have several limitations. In particular, relational databases are built on the assumption of independence between the records, which explains their good performance when querying a set of rows. However, graph data revolves around relationships between data points. This led to the development of native graph databases designed specifically to handle such interconnected data structures.

Neo4j, created by Neo4j, Inc., is the leading native graph DSS, known for its extensive feature set that supports diverse graph-based applications. Neo4j uses a proprietary querying language called *Cypher* to query data stored in a Neo4j database. To boost performance, Neo4j's processing engine relies on a special indexing strategy called [index-free adjacency](#). It works by assigning each node a direct reference to its adjacent nodes,

which implies that accessing relationships and associated data is as simple as a memory pointer lookup. In addition, Neo4j supports simple and composite node and link attributes indexes, constraints, and atomic transactions, as well as functions, subqueries, patterns, and clauses.²⁶ For more advanced graph algorithms and machine learning tasks, the [Neo4j Graph Data Science library](#) is available.

NOTE

A major challenge with graph DSSs is scalability. Solutions such as Neo4j can scale to handle billions of nodes and links, which is more than enough for the vast majority of use cases. However, sharding is not native to graph databases. Unlike relational databases, where rows are independent, graph data is interconnected, making it difficult to partition efficiently as graph density increases.

Mathematically speaking, the problem of partitioning a graph dataset across a set of nodes is near impossible (NP-complete) to perform for large graphs.²⁷

Nevertheless, several graph database technologies (e.g., TigerGraph) have made progress along this way and offer distributed graph processing capabilities.

Another implementation of graph storage systems is managed services. One example is *Amazon Neptune*, which offers a scalable, secure, and cost-efficient managed graph database. Neptune allows users to query data using the popular graph query languages Apache TinkerPop Gremlin, W3C's SPARQL, and openCypher. Another prominent example is *TigerGraph*, which provides a rich platform for native graph data storage and analysis at scale.

Financial use cases of graph databases

Graph databases have been applied to solve a variety of problems in the financial sector. One prominent use case is fraud detection. Many forms of financial fraud exist: credit card fraud, loan fraud, wire fraud, tax fraud, identity theft, insurance fraud, and money laundering. In simple scenarios, fraud is easy to detect with traditional tools and databases. However, in today's environment, fraudsters use sophisticated techniques and tricks to commit their crimes and hide their actions and identities.

The best way to detect such complex patterns is by designing a framework that combines network analysis with a graph database to build a [fraud graph](#). Such a graph records the connections between the actors, transactions, and other pertinent data to help experts capture anomalous trends in the data and create applications that can identify fraudulent activity. [Figure 8-8](#) naively illustrates the concept. In this example, the real account holder is associated with a unique set of information such as address, email, phone number, SSN, etc. A fraudster, however, might use a variety of addresses, phone numbers, and other pieces of information and combine them in complex ways to hide their activities. When represented as a graph, such hidden relationships can be detected more easily. This task can be represented as an entity resolution problem where the goal is to match nodes that represent the same entity.

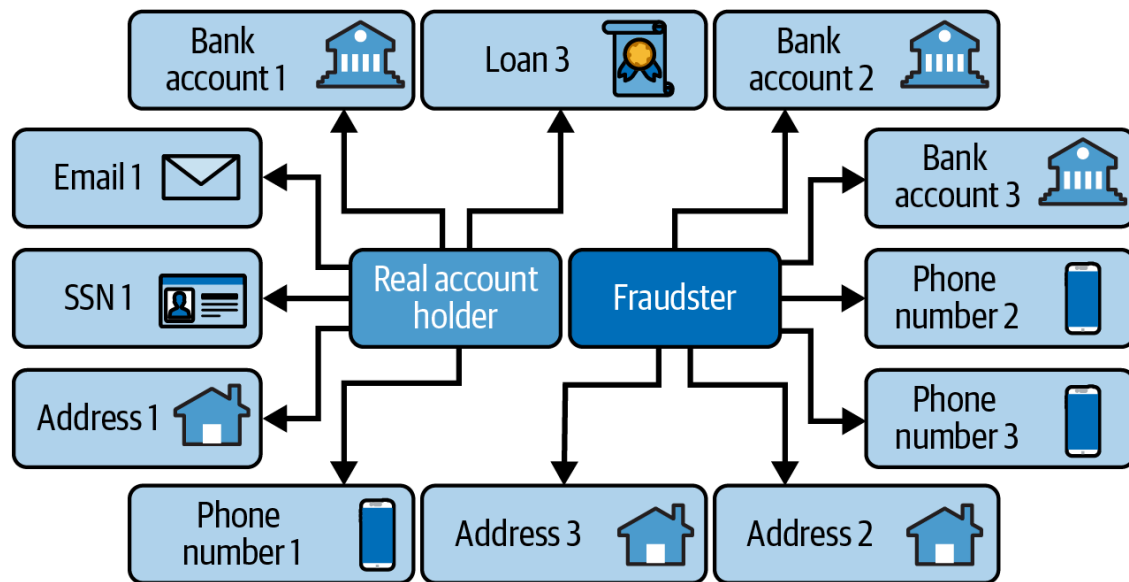


Figure 8-8. Graph-based fraud detection

Graph databases also find applications in financial risk analysis. For instance, financial assets can be structured hierarchically, bundled into new assets, segmented into tranches, and distributed across complex networks of ownership and transactions. This complicates the task of risk management and reduces regulatory oversight. One approach to address this challenge involves constructing a [financial assets graph](#). This graph models and stores the relationships among different types of financial assets, enabling financial institutions to track their assets and account for dependencies in their pricing models. Additionally, graph databases are useful for simulating various shocks and scenarios to test the stability of

correlated asset portfolios, interbank networks, and various types of systemic risks.^{[28](#)}

Banking Circle (BC) is a banking-as-a-service (BaaS) company that offers real-time and low-cost payment services by providing the clients with direct access to clearing in multiple jurisdictions. BC serves a large client base, including over 250 regulated businesses, financial institutions, and marketplaces. It allows clients to move and convert money in real time securely and compliantly. In 2021, BC processed over 250 billion euros in payment volume. Importantly, as the volume of payments that BC processed increased, so did the number of fraud attempts.

To detect fraud, BC initially relied on a traditional rule-based approach, which worked by assigning risk scores by searching and catching certain words, amounts, and locations and sending them to fraud analysts for manual review. However, the company realized that this approach was slow, expensive, and generated a high level of false positives. As a result, it decided to adopt a data-driven AML approach that leverages graph and machine learning techniques. The outcome was a modern framework called SCAM, or System for Catching Attempted Moneylaundering, which consisted of an ensemble of different machine learning models.

To capture complex relationships, SCAM relies on multiple network representations of various data features (e.g., accounts, payments, entities, countries, etc.). Using Neo4j's Graph Data Science (GDS) framework, BC was able to conduct various types of advanced graph analysis.

For example, community detection algorithms were used to generate features that detect high-risk clusters within the network. A community detection algorithm works by finding clusters of similar nodes; for example, if a fraudster uses similar profile attributes, they are likely to fall within the same cluster. This technique, combined with other network features such as risk scores of neighboring nodes and distances to tax havens and known fraudsters, has significantly improved the reliability of SCAM.

BC was able to transform a laborious, sluggish procedure into a data-driven, scalable, flexible solution that leverages cutting-edge technology. False negatives were reduced by 10–25% and the number of overall alerts escalated for manual review was halved. BC continues to experiment

with Neo4j algorithms and plans to add more graph features to tune their model further.

The Warehouse Model

One of the most common business needs in data-driven organizations is an enterprise-wide system for integrated and structured data storage, access, analytics, and reporting. The most popular solution to this need is *data warehousing*, a concept that has been around since the 1970s. Bill Inmon and Ralph Kimball, the two pioneers of this field, define a data warehouse as:

A data warehouse is a subject-oriented, integrated, nonvolatile, and time-variant collection of data in support of management's decisions.

—Bill Inmon

A data warehouse is a copy of transaction data specifically structured for query and analysis.

—Ralph Kimball

I will use the term *warehouse DSM* to describe a system that periodically gathers, consolidates, and structures data from various sources within a company into a central repository intended for various analytical purposes. A DSS system used to implement the warehouse DSM is called a warehouse DSS or data warehouse. [Figure 8-9](#) illustrates a typical data warehouse architecture. As the figure shows, data from heterogeneous sources (left side) get organized into a central data warehouse (middle) which then serves a variety of analytical data needs (right side).

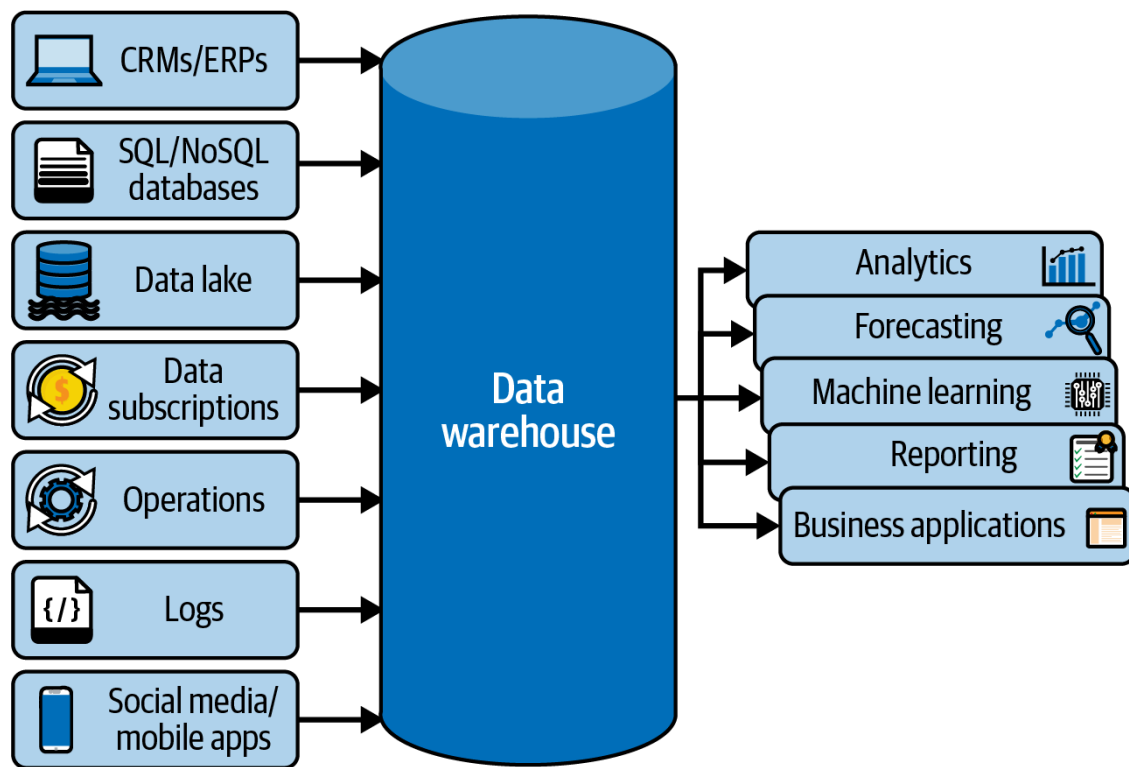


Figure 8-9. A data warehouse architecture

Why data warehouses?

The warehouse DSM offers several advantages, including the following:

Structure

Data warehouses introduce a consistent structure to data, regardless of the original formats provided by the source systems.

Advanced analytics

Performing advanced data queries for business intelligence and reporting is a fundamental and frequent activity within organizations. Data warehouses provide intuitive SQL-like querying capabilities that enable a wide range of data queries.

Scalability

Data warehouses can scale to handle large volumes of data and complex read/write operations.

Subject-oriented

Data warehouses are often designed to allow users to query and analyze data about a particular subject or functional area (such as sales, customers, payments, etc.)

Integrated

Data warehouses consolidate, structure, and harmonize data from different sources (e.g., departments or business units) while ensuring consistency and data quality.

Nonvolatile

Once data is uploaded to a data warehouse, it remains stable and does not change. If any changes occur to a data record, a new record is added instead of updating the existing ones.

Time-variant

Data warehouses record data with a timestamp, allowing users to perform accurate point-in-time historical analysis.

At this point, you might be wondering about the difference between the warehouse DSM and other models that have similar features, such as the data lake or SQL DSMs. Similar to data warehouses, data lakes can also be used to consolidate data into a central location. Data lakes lack default mechanisms to ensure data structure consistency and homogeneity, and they do not offer advanced querying capabilities like a data warehouse does.

The primary difference between a relational DSM and a data warehouse DSM can be understood through the distinction between OLAP and OLTP. The relational DSM is primarily meant for OLAP-oriented applications, emphasizing transactional guarantees and single-row lookups/inserts/updates (often called *Data Manipulation Language*, or DML). The data warehouse DSM is instead aimed toward OLAP-oriented applications that require complex analytical read/write operations with less focus on point lookups and DML. However, it's important to note that both the relational and warehouse models can coexist. In many cases, a significant portion of the data ingested into a data warehouse originates from relational databases that support day-to-day business operations.

Data modeling with data warehouses

When building a data warehouse, data modeling is critical. Over the years, two competing approaches to data modeling have emerged for data warehouses: the *subject modeling approach* of Bill Inmon and the *dimensional modeling approach* of Ralph Kimball.²⁹

In the Inmon approach, data is sourced from various operational systems across the organization and integrated into a centralized data warehouse. Subsequently, based on the specific needs of individual departments or business units, dedicated data marts are derived from the data warehouse. In this model, the data warehouse and the data marts are physically separate entities, each with its own storage and characteristics. Data within the data marts is considered consistent and reliable, as it originates from the centralized source of truth—the data warehouse itself. Users from different departments typically employ specialized tools to access and analyze the data within their respective data marts. [Figure 8-10](#) offers an illustration of the Inmon architecture. The main advantage of this approach is flexibility, as new data marts can be created to meet new business needs. On the negative side, such an architecture introduces a maintenance burden due to the physical separation between the data warehouse and the data marts.

The Kimball approach takes a different perspective. Instead of isolating the data warehouse from the data marts, Kimball suggested a user requirement-driven approach that defines the data marts in advance and integrates them within the data warehouse itself. [Figure 8-11](#) illustrates the concept.

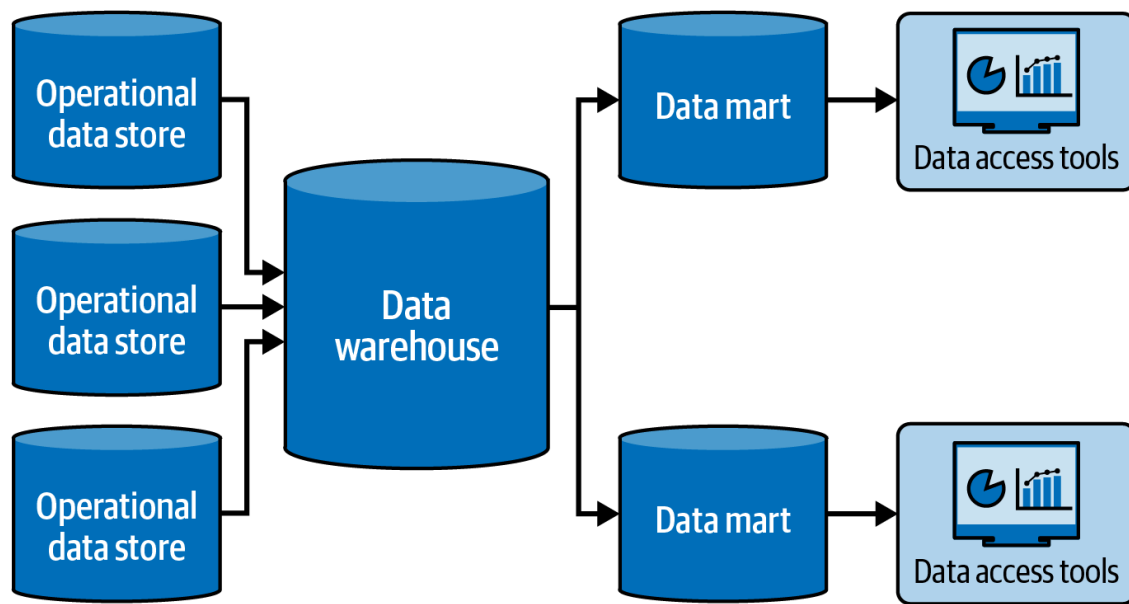


Figure 8-10. Inmon data warehouse architecture

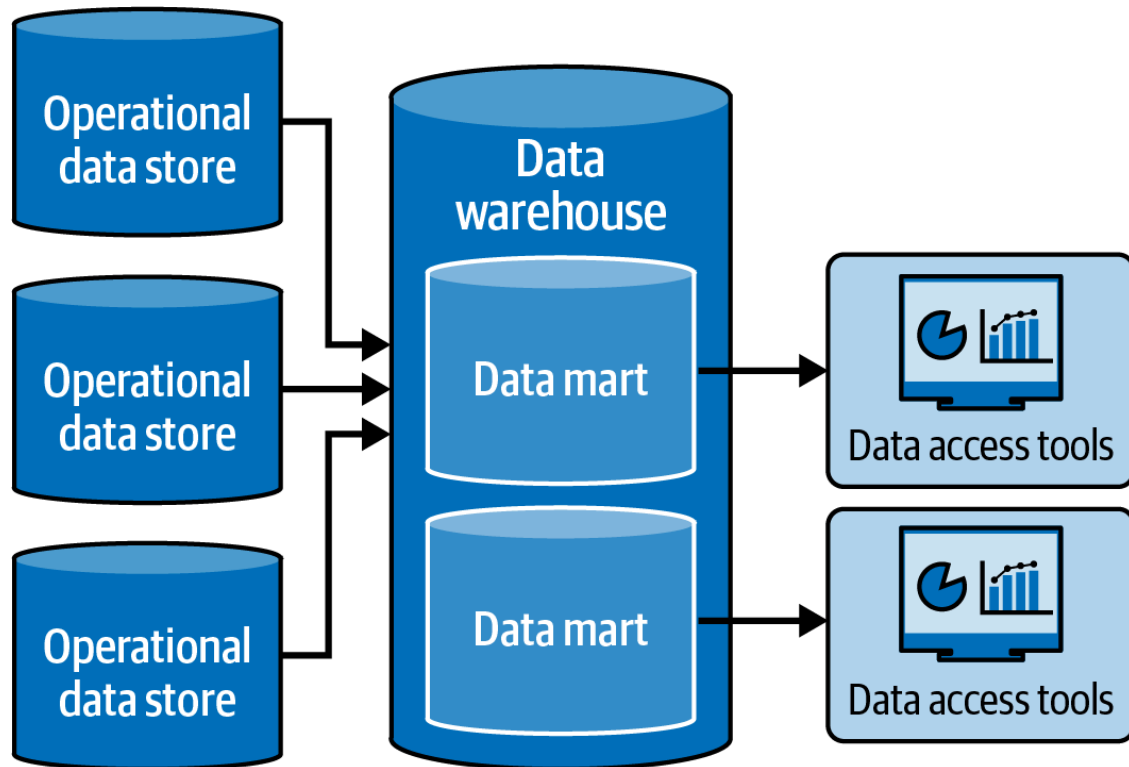


Figure 8-11. Kimball data warehouse architecture

The definition of data marts is done via a technique called *dimensional modeling*. Its main idea is to divide data into *fact* and *dimension* tables. A fact table contains data on quantifiable business entities or events, while a dimension table stores data related to the context of the events. Examples of facts include financial payments, monetary transfers, and online sales. Dimensions may be information such as product, user, and country details. Facts and dimensions may be used to categorize data into

data mart–specific tables based on various business processes such as revenues, products, sales, employees, suppliers, and branches.

In dimensional modeling, two primary types of data warehouse schemas are prevalent: the star schema and the snowflake schema. The star schema features a central fact table connected directly to multiple dimension tables, resembling a star-shaped architecture. On the other hand, the snowflake schema is a normalized version of the star schema that has a single fact table and several direct and indirect dimension tables.

Figure 8-12 illustrates the difference.

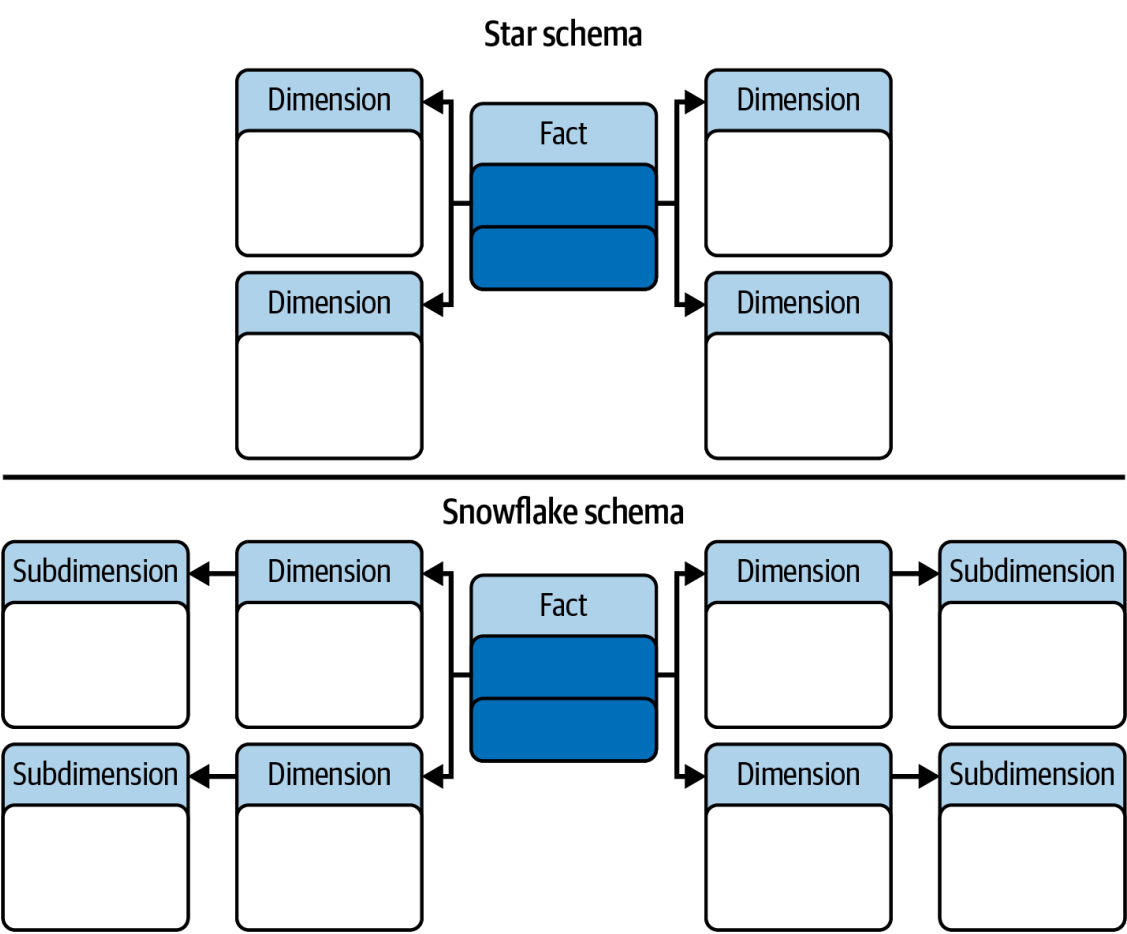


Figure 8-12. Star schema versus snowflake schema

Star schemas are generally easier to understand and simpler to query because they involve fewer table JOIN operations. Snowflake schemas, on the other hand, can be more effective at ensuring data integrity and consistency as they remove redundancy through data normalization.

Technological implementations of data warehousing

Due to the high demand for data warehousing, numerous technological implementations have been developed. As data warehouses emphasize structured data and advanced querying capabilities, most technologies in this domain support SQL and relational data modeling. Leading commercial solutions include high-performance integrated systems like IBM Netezza, IBM Integrated Analytics System (IAS), Db2, Teradata, Oracle Exadata, and Micro Focus Vertica Enterprise. Financial institutions often rely on these solutions for high-performance processing of large datasets.³¹

In recent years, cloud-based data warehousing solutions have seen a notable increase in popularity. Examples include Amazon Redshift, Snowflake, Google BigQuery, and Azure Synapse Analytics. This surge is largely attributed to the convenient features that the cloud offers in terms of scalability, managed infrastructure, on-demand pricing, reliability, and seamless integration with other cloud services. For example, with a data warehousing service such as Google BigQuery, you can easily create a database and begin writing and reading data to it in a matter of minutes. As a serverless platform, BigQuery handles all infrastructure management, so you don't have to worry about provisioning or scaling resources.

Cloud-based data warehouses commonly adopt column-oriented storage, enhancing query performance and reducing costs by retrieving only the columns requested by users. This storage method also benefits from efficiency gains through compression and deduplication techniques, optimizing data storage on disk.³²

Cloud data warehouses achieve scalability by decoupling the compute and storage layers. For example, using Snowflake as a data warehouse, your data will be stored and managed in [separate persistence storage \(e.g., AWS S3\)](#). In contrast, compute resources are virtualized and can be independently scaled up or down by the user, separate from storage.

To enhance storage efficiency and query performance, cloud warehouses leverage the concept of data *partitioning* and *clustering*. The idea is to split a table into a set of storage units called partitions that can be man-

aged and queried separately. With clustering, data within each partition is physically ordered into small blocks of data. When a query is executed against a table, the database engine will first filter the partitions to scan (partition pruning) and then apply block filtering (pruning) to each partition. This way, only the minimum amount of data is queried, thus increasing performance and reducing query cost. BigQuery [requires the user to configure the partitioning and clustering keys \(from existing columns\)](#), while Snowflake implements a more dynamic and managed approach to partitioning and clustering called [*micro-partitioning*](#).

NOTE

When using cloud data warehouses, it's crucial to consider vendor-specific limits, quotas, and usage guidelines. For instance, as OLAP systems, data warehouses are optimized for running complex analytical queries with moderate frequency.

Attempting to support thousands of concurrent users may exceed maximum connection limits. For example, BigQuery allows up to 100 concurrent connections for interactive queries, while Snowflake, with auto-scaling, supports up to 80 concurrent connections. Furthermore, while data warehouses do support data updates and DML operations, they are not primarily designed for these tasks.

Financial use cases of data warehouses

Data warehouses are extensively used in the financial sector to address various business needs. One major use case is the consolidation of data from diverse business silos (e.g., risk, revenue, loans) into a unified data warehouse that serves as the primary source for analytics. This consolidation facilitates the extraction of valuable business insights from the vast volumes of data generated through daily operations within financial institutions. For instance, this enables business teams to effectively understand and manage different types of risks such as credit, financial, operational, and compliance risks. In [Chapter 5](#) on financial data governance, we discussed how data aggregation and consolidation capabilities have become a regulatory requirement for banks following the crisis of 2007–2008.

Data warehouses can enable financial institutions to track financial, operational, and data quality indicators over time. Maintaining point-in-time historical data is a common business requirement in financial markets. A well-designed data warehouse may ensure this by enforcing an append-only policy, which allows new data to be written while maintaining the immutability of existing data. This principle also applies to externally sourced financial data. For instance, financial vendor data is intended for reading and analysis rather than modification, making a data warehouse an ideal solution for such use cases.

Furthermore, data warehouses are widely used in financial markets for facilitating data sharing. For example, various financial data vendors now offer the option to deliver their data to clients via the cloud. This delivery mechanism is particularly convenient with cloud-based data warehouses. Refer to the section [“Data Ingestion Technologies”](#) to learn more about this topic.

BlackRock is the world's largest and leading investment management firm and a provider of financial technology. As of 2023, BlackRock's assets under management reached around 8.5 trillion US dollars. BlackRock's jewel technology is known as *Aladdin*, an integrated solution that combines sophisticated risk analysis features with portfolio management, trading, and operations tools in a single platform. Aladdin has established itself as one of the most prominent examples of financial technologies. In particular, during the 2008 financial crisis, Aladdin showed its remarkable capabilities by allowing companies like Microsoft to aggregate their risk exposures to different banks (a feature that many financial institutions lacked at the time).

To enhance Aladdin's data-driven capabilities, BlackRock launched *Aladdin Data Cloud*, a managed data solution that allows users to combine Aladdin portfolio data with non-Aladdin data, perform timely analysis, and develop custom dashboards and applications using Aladdin Studio—BlackRock's platform for developers. To this goal, BlackRock decided to partner with the cloud-based data warehousing company Snowflake. The main reasons for choosing Snowflake were performance, scalability, and concurrency management.

By bringing together Aladdin, a leader in investment management technology, and Snowflake's Data Cloud, the Aladdin Data Cloud is set to allow its clients to expand the range of data-driven applications across their organizations. Each Aladdin Data Cloud client receives an independent, centrally managed data store preloaded with rich front-to-back Aladdin datasets, which can then be supplemented with proprietary and other third-party data sources, allowing organizations to access and query their business-critical data on a single, cloud-based platform.

Snowflake has several features that make it attractive for building data applications. For example, it isolates work environments via virtual warehouses, which are clusters with dedicated CPU, memory, and temporary storage. In a simple setup, each client can be dedicated to a separate virtual warehouse. Virtual warehouses offer the flexibility to scale resources up and down, thus customizing work environments for different client

needs. In addition, Snowflake has advanced and secure [data-sharing functionalities](#), making it ideal for working with a large base of users and accounts. Moreover, Snowflake developed [its own SQL language](#) with rich command features and SQL standard-compliant syntax. Additionally, Snowflake provides a [list of configurable parameters](#) that can be tuned to control the behavior of a user account, queries, sessions, and objects.

The Blockchain Model

Blockchain is one of the most promising technological trends in today's financial services landscape. It is the underlying technology that enables the so-called *cryptocurrencies*, such as Bitcoin and Ethereum.³³ In this section, I will briefly walk through the basic idea behind blockchain and its applicability as a data storage system for financial data.³⁴

At the most basic level, a blockchain is a data structure that stores data as a chain of linked information blocks. The most prominent application of blockchain technology is in the creation of *digital ledgers*. A digital ledger is a record-keeping book of accounts that keeps track of a business's financial transactions. Think, for example, of when you deposit or withdraw money from your bank account. As per common sense, you trust that your bank will maintain a correct and truthful ledger and not modify it by deleting or updating book records.

A blockchain data ledger performs a similar function and guarantees data immutability, but following different design principles. To illustrate the idea, let's represent a blockchain as a linked list of information blocks, as illustrated in [Figure 8-13](#). The first block in a blockchain is called the *genesis block*. Each subsequent block contains its own data (e.g., transaction records) as well as a *hash pointer* that stores both a pointer to the previous block and a hash of that older block. A hashing function, $H()$, is used to create a cryptographic hash of each block's data. A block hash serves as a unique identifier of each block.

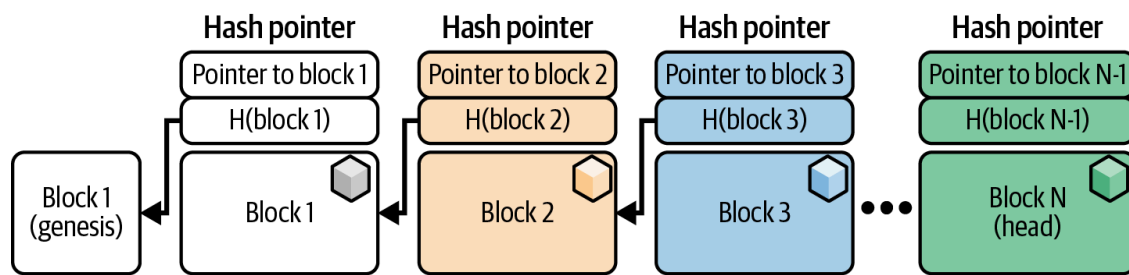


Figure 8-13. Blockchain as a linked list

To illustrate why this structure is tamper resistant, consider an adversary attempting to manipulate the blockchain by altering data entries in block 2. This will automatically update the block's hash. As a result, the hash pointer in block 3 will no longer correspond to the updated hash of block 2. To conduct a successful attack, the adversary must alter all hash pointers up to the most recent block in the list (i.e., the head).

In addition to data immutability, blockchain-based data ledgers have shown high reliability when implemented as a distributed system. The term *distributed ledger technology* (DLT) is often used to describe these systems. In a DLT, a decentralized network of nodes cooperatively maintains the integrity of the ledger. Each node keeps a copy of the blockchain and can verify and validate any operation that alters the blockchain. For example, if a node wants to add a new block to the chain, the other nodes in the network need to agree and reach a consensus on such an operation. The same applies to our earlier example of the adversary attack: even if the adversary were able to change all the hash pointers, the changes would still need to be validated and accepted by the other nodes in the network. A variety of consensus mechanisms are available, including Proof of Work, Proof of Stake, and Byzantine Consensus.

As a financial data engineer, you might be wondering if blockchain is a good solution as a data storage system. Technically speaking, a blockchain can be used to store financial transaction data, but it comes with a price. First, blockchains have very limited data querying capabilities. Second, as more nodes are added to the network, performance may get worse in terms of throughput, latency, or capacity.³⁵ For example, the decentralized nature of blockchain introduces latency in both storing and retrieving data. In applications requiring fast data access, the time taken for

reaching consensus, propagating blocks, and verifying transactions can lead to slower response times.

To overcome these issues, blockchain-based databases (or blockchain databases for short) were introduced. The main idea behind a blockchain database is to combine the best of both worlds: the performance and power of a traditional database system (e.g., SQL or document databases) with the immutability, cryptographic verifiability, integrity, decentralized control, and transaction traceability of a blockchain.

Crucially, when deployed internally within a financial institution, a blockchain database doesn't need to be decentralized. The institution in this case acts as the administrative authority that controls the blockchain.³⁶

A few commercial solutions for blockchain databases already exist. The most prominent is [BigchainDB](#), which uses MongoDB as the distributed database under the hood and offers blockchain characteristics. Another example is [Amazon Quantum Ledger Database \(QLDP\)](#), a fully managed ledger database for creating an immutable, transparent, and cryptographically verifiable transaction log.

A successful story in the commercial implementation of blockchain technology is Ripple. Behind this is an American company called Ripple Labs, Inc. It is a unique player in this industry as it created the first financial services platform and network that leverages blockchain, tokenization, and cryptocurrency technologies for enterprises. Ripple's primary use case is to enable secure, instant, and low-cost cross-border financial transactions and settlements.

Ripple's primary offering is RippleNet, a blockchain-based infrastructure for payments. As of the end of 2024, RippleNet connects a network of over 500 participants.

XRP is Ripple Labs's native cryptocurrency (digital asset). It is used as a bridge currency in Ripple's ecosystem to facilitate liquidity for cross-border transactions. XRP is independent of RippleNet but can be utilized within the network for certain services. XRP is a [bridge asset](#), or an asset that businesses and financial institutions can use to make a bridge transfer between two different fiat currencies. In this scenario, a financial institution can purchase an equivalent amount of XRP and send it through Ripple's network.

The underlying blockchain ledger technology that powers Ripple is called XRP Ledger (XRPL). Interestingly, [XRPL differs from Bitcoin's blockchain](#). Bitcoin relies on an energy-intensive Proof-of-Work (PoW) mechanism for transaction validation, while Ripple uses the faster and more efficient mechanism known as the [XRP Ledger Consensus Protocol](#), or XRP LCP for short. For an intuitive introduction to XRP LCP, see the [CoinBrain website](#). This consensus mechanism is key to XRPL's efficiency, enabling rapid transactions at low costs. XRPL has been released to the public, and it is now an [open source blockchain protocol](#).

To illustrate an example (sourced from the [Ripple website](#)), let's say that Bank A intends to transfer \$10,000 to Bank B in euros using the Ripple platform. The banks agree on a Forex rate on Ripple, following which Bank A converts the USD to XRP at the agreed rate and transfers it via the XRP Ledger to Bank B. Upon receipt, which happens in a matter of sec-

onds, Bank B converts the XRP to euros, with minimal transaction fees compared to traditional banking methods. This use of XRP as a bridge currency via RippleNet demonstrates its role in facilitating fast and cost-effective cross-border transactions.

Ripple has helped countries create their own [central bank digital currencies \(CBDCs\)](#) through its Ripple CBDC platform. Furthermore, Ripple is a member of the ISO 20022 Standards Body, becoming the first member of the ISO organization dedicated to Distributed Ledger Technology (DLT).

In conclusion, blockchain presents a complex landscape with ongoing efforts aimed at exploring its feasibility for high-storage and high-performance applications. Researchers and developers are actively investigating various approaches to enhance blockchain's capabilities, such as sharding, optimizing consensus algorithms, Layer 2 protocols, and sidechains, and are exploring hybrid architectures that combine blockchain with traditional databases.

Summary

This chapter covered the second layer of the financial data engineering lifecycle: storage. This layer facilitates the choice and implementation of various data storage models and systems, supporting the storage and retrieval of data within a financial data infrastructure.

Crucially, data is not merely ingested and stored. To unlock its full potential in driving informed decision-making and operational excellence, data must undergo a systematic transformation into structures that align with the diverse needs of stakeholders within financial institutions. This takes us to the next layer: transformation and delivery. This layer serves as the bridge between raw data ingestion and its utilization by end users.

Let's explore this critical layer further in the next chapter.

- 1** For a comprehensive overview of data modeling, I highly recommend Graeme Simsion and Graham Witt's *Data Modeling Essentials*, 3rd ed. (Morgan Kaufmann, 2004).
- 2** This definition is derived from the one provided by PostgreSQL [documentation](#).
- 3** The acronym ACID was coined in 1983 by Andreas Reuter and Theo Härder in their seminal work, "[Principles of Transaction-Oriented Database Recovery](#)", *ACM Computing Surveys (CSUR)* 15, no. 4 (December 1983): 287–317.
- 4** The definition I present for snapshot isolation is provided in an operational way, which is done for the sake of simplicity. This concept can be further explored from a more rigorous, mathematical point of view; for example, see Andrea Cerone and Alexey Gotsman's "[Analysing Snapshot Isolation](#)", *Journal of the ACM (JACM)* 65, no. 2 (January 2018): 1–41.
- 5** This example was inspired by the excellent [illustration of the CAP theorem by ByteByteGo on YouTube](#).
- 6** To learn more about consistency tradeoffs, I highly recommend Daniel Abadi's "[Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story](#)", *Computer* 45, no. 2 (February 2012): 37–42.
- 7** To learn more about database stress testing, see Bert Scalzo's *Database Benchmarking and Stress Testing: An Evidence-Based Approach to Decisions on Architecture and Technology* (Apress, 2018).
- 8** See, for example, "[How We Stress Test and Benchmark CockroachDB for Global Scale](#)", by Stan Rosenberg, Alexander Shraer, William Kulju, and Alex Lunev.
- 9** For more on this topic, read Yishan Li and Sathiamoorthy Manoharan's "[A Performance Comparison of SQL and NoSQL Databases](#)", in the *2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)* (IEEE, 2013): 15–19.
- 10** For more details, see Rick Cattell's "[Scalable SQL and NoSQL Data Stores](#)", *ACM SIGMOD Record* 39, no. 4 (May 2011): 12–27.
- 11** For example, Netflix created an orchestration engine called Conductor which uses Dynamite for primary persistence storage. On top of that, Elasticsearch is used as

a secondary indexing database to allow workflow search and discovery. For more on Conductor, read the [official documentation](#).

- 12** Example inspired by the excellent article [“Native vs. Non-Native Graph Database”](#), by John Stegeman.
- 13** For more details on this data lake architecture pattern, I recommend reading Franck Ravat and Yan Zhao’s [“Data Lakes: Trends and Perspectives”](#), in *Database and Expert Systems Applications: 30th International Conference, DEXA 2019, Proceedings, Part I* (August 2019): 304–313.
- 14** To learn more about the history of SQL, I highly recommend Donald D. Chamberlin’s [“Early History of SQL”](#), *IEEE Annals of the History of Computing* 34, no. 4 (October–December 2012): 78–82.
- 15** For a good read on this topic, see [“Anomalies in Relational Model”](#) by GeeksForGeeks.
- 16** For an excellent illustration of these normal forms with examples, I suggest checking the article [“Normalization in DBMS”](#) by Study Tonight.
- 17** For a good overview of SQL database indexing, see the official PostgreSQL [documentation on indexes](#).
- 18** For a good comparison between relational database management system features, consult [Wikipedia](#).
- 19** A detailed account of this problem is offered in the blog post [“Why Sharding Is Bad for Business”](#), by Michelle Gienow.
- 20** For a general overview, consult the official MongoDB [page on indexing](#).
- 21** To learn more, read InfluxDB’s documentation on the [InfluxDB storage engine](#).
- 22** For a comparative study on this topic, I recommend Fazl Barez, Paul Bilokon, and Ruijie Xiong’s [“Benchmarking Specialized Databases for High-frequency Data”](#), *arXiv preprint*; arXiv:2301.12561 (January 2023).
- 23** In asynchronous communication, an application sends a request or message in a *fire-and-forget* mode, meaning that a response isn’t expected from the target receiver or consumer.

- 24** For more on this topic, consult the [Confluent Developer website](#).
- 25** For an interesting real-world application of this idea, I highly recommend watching the [presentation given by two AWS experts](#) on how to build a real-time financial data feed as a service on AWS.
- 26** For more details on these features, consult the official [Neo4j documentation page](#).
- 27** To learn more about this topic, see David Montag's [“Understanding Neo4j Scalability”](#) white paper, *Neotechnology* (January 2013).
- 28** An example of a company that offers such services is [Financial Network Analysis \(FNA\)](#).
- 29** For a comprehensive comparative study of the difference between Inmon and Kimball models, see Lamia Yessad and Aissa Labiod's [“Comparative Study of Data Warehouses Modeling Approaches: Inmon, Kimball and Data Vault”](#), in 2016 *International Conference on System Reliability and Science (ICSRS)* (IEEE, 2016): 95–99.
- 30** For an excellent reference on dimensional modeling, I recommend Ralph Kimball and Margy Ross's *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. (Wiley, 2013).
- 31** For instance, the NYSE transitioned to IBM Netezza to manage its growing data volumes, which traditional SQL databases were unable to handle effectively. Read more in [“At NYSE, the Data Deluge Overwhelms Traditional Databases”](#), by Tom Groenfeldt.
- 32** For a deeper understanding of how and why columnar formats are used in data warehouses, I highly recommend reading about Google BigQuery's columnar storage format [Capacitor](#).
- 33** For a good introduction to cryptocurrencies, I highly recommend Arvind Narayanan, Joseph Bonneau, Edward Felten, Andrew Miller, and Steven Goldfeder's *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction* (Princeton University Press, 2016).
- 34** For a deeper introduction to blockchain databases, I highly recommend Chapter 26 of Avi Silberschatz, Henry F. Korth, and S. Sudarshan's 2019 *Database Systems*

Concepts, 7th ed. (McGraw Hill, 2019).

35 For more on performance-related issues in blockchains, read Mayank Raikwar, Danilo Gligoroski, and Goran Velinov's ["Trends in Development of Databases and Blockchain"](#), in the *2020 Seventh International Conference on Software Defined Systems (SDS)* (IEEE, 2020): 177–182.

36 To learn more about blockchain database design patterns, I recommend MongoDB's guide ["Blockchain Database: A Comprehensive Guide"](#).